Analyze Your Leakage! Security Analysis of Encryption Schemes for Substring Search

Zichen Gui
University of Georgia

Kenneth G. Paterson
ETH Zürich

Sikhar Patranabis
IBM Research India

January 17, 2026

Abstract

Searchable symmetric encryption (SSE) enables queries over symmetrically encrypted databases. To achieve practical efficiency, SSE schemes incur a certain amount of leakage; however, this leads to the possibility of leakage cryptanalysis, i.e., cryptanalytic attacks that exploit the leakage from the target SSE scheme to subvert its data and query privacy guarantees. Leakage cryptanalysis has been widely studied in the context of SSE schemes supporting either keyword queries or range queries, often with devastating consequences. However, little or no attention has been paid to cryptanalysing substring-SSE schemes, i.e., SSE schemes supporting arbitrary substring queries over encrypted data. This is despite their relevance to many real-world applications, e.g., in the context of securely querying outsourced genomic databases.

In this paper, we present the *first* leakage cryptanalysis of three prominent substring-SSE schemes due to Chase and Shen (PoPETS '15), Faber *et al.* (ESORICS '15), and Hahn *et al.* (SIGMOD 2018). We propose novel, inference-based *query reconstruction attacks* on each of these schemes that exploit their respective leakage profiles. We implement our attacks and experimentally validate their success rates and efficiency over real-world datasets. Our attacks achieve high query reconstruction rate with practical efficiency, and scale smoothly to large datasets. Notably, our attack success attack rate is the highest against the scheme due to Hahn et al. (SIGMOD 2018), which is the most recent of the three schemes. This demonstrates that, as far as substring-SSE is concerned, newer schemes are not necessarily "more secure".

To the best of our knowledge, ours are the first and only query reconstruction attacks on (and the first systematic leakage cryptanalysis of) *any* substring-SSE scheme to date. Query reconstruction against substring-SSE schemes is inherently harder to achieve than against traditional SSE schemes for keyword search, as the query space for substring-SSE is significantly larger. In fact, while the vast majority of known attacks against SSE schemes for keyword search restrict the query space to the most-frequent keywords, our attacks achieve decent query recovery rates *without* any such restriction on the queried substrings.

Our work carries a wider message for designers of SSE schemes: it is not sufficient to merely identify the leakage profile of any new scheme; rather, it is incumbent on designers to cryptanalyze that leakage profile and show that is not consequential in the face of state-of-the-art attacks. The novel attack techniques in our paper provide a starting point for such analysis for substring-SSE schemes.

Contents

1 Introduction

Searchable Symmetric Encryption. Searchable symmetric encryption (SSE) [9, 12, 15, 25, 67] supports efficient queries over symmetrically encrypted databases. SSE is a key enabler for secure storage-as-a-service, wherein clients can securely outsource the storage and processing of large databases to (potentially untrusted) third party servers. The goal of SSE is to enable efficient query processing *directly* over the encrypted database, while ensuring client privacy by minimizing information "leakage" to the server.

The vast majority of SSE schemes in the literature [5, 6, 10–12, 15, 25, 43, 67] support *single keyword* queries (i.e., given an encrypted document collection in which each document is tagged with keywords, it is possible to find the set of all documents associated with a target keyword). SSE schemes supporting a richer class of Boolean queries over collections of keywords have also been studied [8, 9, 40, 51, 61, 63]. Other works have investigated SSE schemes for range queries [17–20], as well as for join and group-by queries over encrypted relational databases [17, 39, 41].

Substring-SSE. In this paper, we focus on *substring-searchable symmetric encryption* (substring-SSE) that supports the following query functionality: given an encrypted string (where the string is a sequence of characters picked from some fixed alphabet), return the positions of all occurrences of a target substring within this string. As a motivating use-case, suppose that a medical research lab wishes to store the genomic data of subjects on a cloud server, while allowing researchers to issue substring queries to detect the existence of particular DNA sequences (e.g., to trace the existence of cancer marker sequences, or to determine the rarity of a potentially useful probe sequence). For such an application, substring-SSE would offer a solution that ensures the privacy of the database as well as the queries.

The naïve approach of re-purposing SSE for single keyword search to construct substring-SSE (where each substring of a given string is modeled as a separate keyword) is practically inefficient: for a length n string, the number of keywords required is $O(n^2)$. Achieving practically efficient substring-SSE is much more challenging and seems to require very different techniques from those underlying SSE for single-keyword search. This is exemplified by the three most prominent and widely cited works in this direction [13, 20, 36]. These schemes broadly represent three fundamentally different techniques underlying many practically efficient substring-SSE schemes. Indeed, several other substring-SSE schemes [52, 69, 71–73] can be viewed as refinements/extensions of [13] that attempt to improve search complexity and/or storage efficiency by using variants of the data structures used in [13].

Certain other substring-SSE proposals [4, 56] use heavier cryptographic tools, such as Oblivious RAM (ORAM) [1, 10, 17, 22, 26, 27, 66, 68], Private Information Retrieval (PIR) [14, 37, 49, 55], or Fully Homomorphic Encryption (FHE) [23]. Such schemes are impractical at meaningful scale. [57] proposed an extension of standard SSE for single-keyword search that additionally allows querying substrings of keywords. This is a different substring-search functionality as compared to the notion of substring-SSE we consider in this paper. See [53] for a comprehensive survey of substring-SSE schemes.

Leakage in SSE. For any SSE scheme, "leakage" denotes the information that the server learns about either the database itself or the client's queries. Existing SSE designs opt for enhanced performance at the cost of leaking some information to the server [9, 12, 15]. A typical security proof for such a scheme establishes, via a simulation-based security model, that the scheme in operation leaks no more than a certain well-defined leakage function. This calls for an assessment of the real-world impact of that leakage, via *leakage cryptanalysis*, which involves developing concrete cryptanalytic attacks that exploit the leakage to subvert some security guarantee (such as data or query privacy). Leakage cryptanalysis has been studied extensively in the context of SSE for keyword queries [3,7,16,32,33,38,58–60,64] and SSE for range queries [21, 29–31, 44–48, 50, 54]. Such attacks typically exploit one or more forms of leakage that commonly occur in SSE schemes. A large fraction of these attacks target *query reconstruction*.

Leakage in Substring-SSE. To the best of our knowledge, *no* prior work has carried out any leakage cryptanalysis of substring-SSE schemes. In particular, *none* of the works that have proposed substring-SSE schemes [4, 13, 20, 36, 52, 56, 57, 69, 71–73] have attempted to cryptanalyse the leakage of their own proposals.

Challenges. We emphasize here that attempting to cryptanalyse the leakage from substring queries throws up new challenges that do not arise in the context of SSE for keyword or range queries. First, unlike in the case of keyword queries, where the set of potential keywords is fixed *a priori* (and is typically in the range of millions for even large real-world document collections), arbitrary substring queries come from a significantly larger universe (for example, the number of possible strings of length 5 containing characters from the English alphabet is more than a billion). This limits the usefulness of known leakage cryptanalysis techniques on SSE for keyword search in the context of substring-SSE.

Second, the query equality leakage from keyword queries is essentially "all-or-nothing" – either two keywords are identical or they are not. This is particularly useful in many existing attacks that rely on query equality to (informally speaking) *filter* the potential set of keywords matching a given query [3, 7, 33]. However, in the case of substring queries, two substrings may have varying degrees of overlap in terms of the number of characters they have in common. This leads to the possibility of more nuanced, fine-grained query equality leakage in substring-SSE that needs more advanced filtering approaches.

Given these novel analysis challenges that are unique to substring-SSE, the lack of prior cryptanalysis across *all* existing substring-SSE proposals, and the central role of such analysis in assessing the real-world security of SSE schemes (as is widely acknowledged for SSE targeting other query types), we believe that leakage cryptanalysis of substring-SSE schemes is a problem deserving of attention.

1.1 Our Contributions

We present the *first* leakage cryptanalysis of of three prominent substring-SSE schemes due to Chase and Shen [13] (henceforth, the CS scheme), Faber *et al.* [20] (henceforth, the FJK+ scheme), and Hahn *et al.* [36] (henceforth, the HLK scheme). More specifically, we propose

novel query reconstruction attacks against each of these schemes. We implement our attacks and experimentally validate their success rates and practical efficiency over two real-world datasets – English Wikipedia[1] and the Enron Email Corpus[2]. Our attacks achieve high query reconstruction rate with practical efficiency, and scale smoothly to large datasets.

To the best of our knowledge, ours are the *first and only* query recovery attacks on (and the first systematic leakage cryptanalysis of) any substring-SSE scheme. We focus on the CS, FJK+ and HLK schemes because: (i) they are the three most prominent and widely cited works in this direction, (ii) they broadly represent three fundamentally different techniques underlying the vast majority of practically efficient substring-SSE schemes *without* the usage of heavy cryptographic machinery (and hence, attacks on these schemes are likely to be extensible to other practically efficient substring-SSE schemes that build upon these three schemes and incur similar leakage), (iii) they represent the handful of works on substring-SSE that have appeared at reputed, peer-reviewed publication venues, and (iv) they are featured most prominently in the only (to the best of our knowledge) comprehensive survey on substring-SSE [53] (see, in particular, Table 2 of [53]).

We remark that each of the CS, FJK+ and HLK schemes use very different techniques, and thus have incomparable leakage profiles. The diverse nature of the techniques and leakage profiles for these schemes seemingly makes a universal attack strategy against all of these schemes impossible. As a result, we have to design distinct attacks that specifically exploit the leakage from each target scheme. Notably, our attack success attack rate is the highest against the HLK scheme [36], which is the most recent of the three schemes. This demonstrates that, as far as substring-SSE is concerned, newer schemes are not necessarily "more secure".

As mentioned earlier, query reconstruction against substring-SSE is inherently harder to achieve than against traditional SSE schemes for keyword search, as the query space for substring-SSE is significantly larger. To overcome this challenge, we develop several novel optimization and approximation techniques that allow our attacks to be highly scalable. In fact, while the vast majority of known attacks against SSE schemes for keyword search [3, 7, 16, 32, 33, 38, 58–60, 64] crucially rely on restricting the query space to the most-frequent keywords for efficiency and scalability, our attacks on substring-SSE achieve decent query recovery rates and high practical efficiency *without* any such restriction on the queried substrings. We present a more detailed overview of our contributions below.

General Attack Approach. A common principle underlying all of our attacks is as follows: we show how to develop novel statistical models for substring queries based on the leakage components from each target scheme and auxiliary data in the form of an approximate version of the original database. We then use the resulting models to develop new inference-style leakage cryptanalysis-based attacks that target query reconstruction against each of these schemes. We additionally highlight that all of our attacks assume an "honest-but-curious" (passively eavesdropping) server, which is the most commonly assumed adversarial model in the broader SSE literature. See Sections 3.2, 4.2, and 5.2 for finer details of our attacks on the CS, FJK+ and HLK schemes, respectively.

[1] https://dumps.wikimedia.org/simplewiki/
[2] https://www.cs.cmu.edu/~enron/

Auxiliary Data. All of our attacks rely on auxiliary data, that is, additional data that is assumed to be distributed in the same way as that in target database. More precisely, we assume that the target database and the auxiliary database are sampled independently from the same distribution. This constitutes a weaker attack assumption as compared to "known-data attacks" (e.g. [3, 7, 33, 38]), where the auxiliary and target databases are assumed to be identical. The precise sampling strategy used in our experimental evaluation can be summarized as follows: given a dataset, we set aside half of it for use as the attack target, i.e., for generating the leakage, while using sub-samples of the other half to define auxiliary data. In this way, we effectively use independent samples from the same empirical distribution to define the target and auxiliary data distributions. This is akin to the standard approach of separating training data from test data that is used, for example, in machine learning. The approach is described in more detail in Sections 3.3, 4.3 and 5.3.

Implementation. We show how to efficiently implement our attacks against each of the CS, FJK+ and HLK schemes on large sets of queries and leakage. To handle large candidate sets for arbitrary query substrings, we design and implement several computational optimization and approximation techniques. These make our attacks highly scalable with respect to the number of queried substrings, the number of candidate substrings in the auxiliary information, and the size of the target dataset. We develop our own implementation of simulated annealing for speed and flexibility.

Experiments. In Sections 3.3, 4.3 and 5.3, we present extensive experimental evaluations to validate the practicality of our proposed attacks over English Wikipedia and the Enron Email Corpus. For English Wikipedia (100K Wikipedia pages, 10K queries each of length between 3 and 15), we achieve query reconstruction rates of 23%, 57% and 60% against CS, FJK+ and HLK, respectively. The corresponding query reconstruction rates for the Enron Email Corpus (200K emails, 10K queries each of length between 3 and 15) are 19%, 62% and 61%, respectively. For our attack against CS, another interesting success metric is the character recovery rate (i.e., how many characters within the queries are guessed correctly). Our attack achieves 41% and 34% character recovery rate in the two settings above, respectively.[3] The instructions and scripts for running our attacks can be found at https://github.com/substring-sse-attacks/substring-sse-attacks/. All experimental results in the paper can be reproduced with the scripts.

Comparison with Attacks on SSE for Keyword Search. Almost all known query reconstruction attacks against SSE schemes for keyword search (such as in [3, 7, 16, 32, 33, 38, 58–60, 64]) restrict their target query space to the most frequent keywords in the database. Notably, without this limitation, such attacks have significantly lower query reconstruction rates (see [3] for a detailed exposition). On the other hand, for substring-SSE, we demonstrate attacks that achieve high query reconstruction rates without any such restrictions on the query space. As a concrete illustration, for our attacks using the Enron Email Corpus with 200K documents and 10K queries, about 30% of the reconstructed queries have query response volume below 100. For comparison, in the context of SSE for keyword search, the 1000-th most frequent keyword in the Enron Email Corpus (for the same number of documents) occurs in more than 4K documents, and would typically be

[3]The character recovery rate is not as interesting in the context of the FJK+ and HLK schemes, which rely on n-grams.

hard to reconstruct with reasonable success probability using known attacks (such as in [3]). Additionally, designing efficiency-preserving countermeasures against leakage cryptanalysis appears to be significantly more challenging for substring-SSE as compared to SSE for keyword search. See Section 6 for a more detailed discussion.

2 Preliminaries

Plaintext Substring Matching. Databases for plaintext substring matching are modelled as follows. Let $\mathbf{DB} = (S_1, \ldots, S_N)$ be a collection of strings (also referred to as documents) over a shared alphabet Σ, i.e., $S_i \in \Sigma^*$ for all $i \in [1, N]$. We refer to the collection $\mathbf{DB}$ a database. A plaintext substring query $\mathbf{Query} : \Sigma^* \times (\Sigma^*)^* \to \{0,1\}^*$ takes as input a string $q \in \alpha^*$ and the database $\mathbf{DB} = (S_1, \ldots, S_N)$, and outputs a list of index-position pairs $(\mathsf{ind}_i, \mathsf{pos}_i)_{i=1}^l$ such that

$$S_{\mathsf{ind}_i}[\mathsf{pos}_i, (\mathsf{pos}_i + |q|)] = q \quad \forall i = 1, \ldots, l.$$

We say that $\mathbf{Query}$ is correct if $\mathbf{Query}(\mathbf{DB}, q)$ returns all matching index-position pairs of q in $\mathbf{DB}$ for all $\mathbf{DB}$ and q. For simplicity, we write $\mathbf{DB}(q)$ to mean $\mathbf{Query}(\mathbf{DB}, q)$.

Syntax of Substring-SSE. A substring-searchable symmetric encryption (substring-SSE) scheme Π is defined by a tuple of three algorithms $\Pi = (\mathbf{Gen}, \mathbf{Setup}, \mathbf{EQuery})$:

- $\mathsf{sk} \leftarrow \mathbf{Gen}_{\mathbf{Clt}}(1^\lambda)$ is a probabilistic algorithm run by the client $\mathbf{Clt}$. It takes as input a security parameter 1^λ and outputs a secret key sk.
- $(\perp, \mathbf{EDB}) \leftarrow [\mathbf{Setup}_{\mathbf{Clt}}(\mathsf{sk}, \mathbf{DB}), \mathbf{Setup}_{\mathbf{Svr}}()]$ is an interactive protocol between the client $\mathbf{Clt}$ and the server $\mathbf{Svr}$. The client takes as input a secret key sk and a database $\mathbf{DB}$ and the server does not take any input. After the interaction, the server outputs an encrypted database $\mathbf{EDB}$.
- $((\mathsf{ind}_i, \mathsf{pos}_i)_{i=1}^l, \perp) \leftarrow [\mathbf{EQuery}_{\mathbf{Clt}}(\mathsf{sk}, q), \mathbf{EQuery}_{\mathbf{Svr}}(\mathbf{EDB})]$ is an interactive protocol between the client $\mathbf{Clt}$ and the server $\mathbf{Svr}$. The client takes as input a secret key sk and a substring query q and the server takes as input the encrypted database $\mathbf{EDB}$. After the interaction, the client obtains a list of index-position pairs $(\mathsf{ind}_i, \mathsf{pos}_i)_{i=1}^l$ and the server obtains no output.

A substring-SSE scheme should satisfy certain correctness and security properties. We defer detailed descriptions of these properties (and additional discussions around security/attack models) to Appendix A.

3 Query Reconstruction Attack against the CS Scheme

In this section, we describe our query reconstruction attack against the CS substring-SSE scheme proposed by Chase and Shen in [13].

3.1 The CS Scheme

Suffix Trees. We first present an introduction to suffix trees, the primary data structure used in the CS scheme. Let S be a string of length n. A suffix tree T_S for string S is a tree with the following properties: (i) the tree has exactly n leaves, (ii) except for the root, each (internal) node has at least two children, (iii) each edge is labelled with a non-empty substring of S, and (iv) no two edges with the same starting node share the same starting character. Also, let N be an internal node or a leaf node of T_S. The string on the path from the root to node N is defined as the concatenation of strings on the edges from the root to node N. The node N stores the starting position of the first instance of the string on the path from the root to node N in S. From now on, we write $\mathsf{ind}(\mathsf{N})$ to denote the string position stored in node N and $\mathsf{path}(\mathsf{N})$ to denote the string on the path from the root to the node N.

Suffix Tree for Multiple Strings. The suffix tree described above can be generalised to support multiple strings. The resultant data structure is known as a generalised suffix tree [2]. The main difference between a suffix tree and a generalised suffix tree is that in the latter case, the nodes no longer store just the starting positions of the matches. Instead, the nodes store the string indices (i.e., in which strings the substring appears) and the starting positions of the matches. We will still call the content stored in the nodes *index* for convenience. Figure 1 gives an example of a generalised suffix tree for the two strings "hello" and "help".

Although [13] only described how to use their scheme to search over a single string, their scheme can be extended easily to support searching over multiple strings. In our attack, we consider the version of their scheme that supports multiple strings, but our attacks work just as well for the original single-string version.

Substring Query using Generalised Suffix Trees. We now illustrate how to efficiently perform (plaintext) substring matching using a generalised suffix tree representing multiple strings. Consider a substring query "ell". By traversing the suffix tree (Figure 1) with "ell", we will end on node N_9. Although the node N_9 stores the starting index for the suffix "ello", the index is also the starting index of substring "ell" since "ell" is a prefix of "ello". As a result, we obtain the correct query response corresponding to the query "ell" by traversing the suffix tree.

The suffix tree also supports substring queries with multiple matches. To illustrate that, consider a substring query on "he". Here, instead of retrieving the index stored in node N_2, one can retrieve all the indices stored in the leaf nodes of the subtree rooted at node N_2. Since these leaf nodes contain all the starting indices of suffixes starting with "he", retrieving all indices stored in the leaf nodes yield all matches of "he".

The CS Scheme. We now provide a simplified description of the CS scheme due to space constraints. The full description of the CS scheme can be found in Appendix B.

Define the initial path of node N, written as $\mathsf{initpath}(\mathsf{N})$, as the concatenation of strings on the edges from the root to node N, except that for the last edge (closest to N), only the first

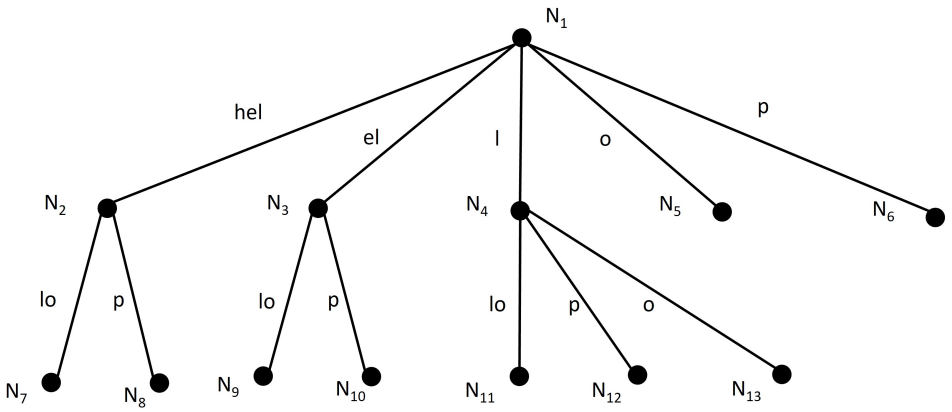

Figure 1: The generalised suffix tree for "hello" and "help". The nodes in the tree store the index of the first occurrence of the string from the root node (N_1) to the current node. For example, N_2 stores $(1,1)$ since the first occurrence of "hel" occurs at the first position of the first string. The node N_{12} stores $(2,3)$ since "lp" occurs at the third position in the second string. Some examples for the notations used: $\mathsf{ind}(\mathsf{N}_7) = (1,1)$; $\mathsf{path}(\mathsf{N}_7)$ = "hello"; $\mathsf{initpath}(\mathsf{N}_9)$ = "ell"; $\mathsf{leafpos}(\mathsf{N}_3) = 3$; $\mathsf{num}(\mathsf{N}_3) = 2$.

character of the string on the edge is used. For example, in Figure 1, $\mathsf{initpath}(\mathsf{N}_7)$ = "hell" and $\mathsf{initpath}(\mathsf{N}_9)$ = "ell".

In the CS scheme, the encrypted database $\mathbf{EDB}$ has entries of the form

$$\mathbf{EDB}[\mathsf{F}_{\mathsf{sk}_1}(\mathsf{initpath}(\mathsf{N}))] = \mathsf{Enc}_{\mathsf{sk}_2}(\mathsf{ind}(\mathsf{N})),$$

where F is a PRF and Enc is an encryption algorithm. Then, suppose the client wants to make a query q, it sends $\mathsf{F}_{\mathsf{sk}_1}(q[1,1]), \mathsf{F}_{\mathsf{sk}_1}(q[1,2]), \ldots, \mathsf{F}_{\mathsf{sk}_1}(q[1,|q|])$ to the server, where $q[a,b]$ denotes the substring from position a to position b in q. The server checks if any of the PRF values exists in $\mathbf{EDB}$ and returns the match that corresponds to the PRF value containing the longest queried string.

Of course, this only allows the client to learn the first match of the query, since $\mathsf{ind}(\mathsf{N})$ only stores the first occurrence of $\mathsf{path}(\mathsf{N})$. Chase and Shen proposed modifications to the construction above so that the query returns all matching indices.

3.2 Our Attack against the CS Scheme

Notable Leakage Components. For completeness, we give a full description of the leakage profile of CS in Appendix C.1. Our attack exploits the following leakage components of the scheme.

- *Length of the initial path:* Our attack aims to recover the initial paths of the queries. The length of the initial path, or $\mathsf{ipLen}_i = |\mathsf{initpath}(q_i)|$ is useful for the attacker to restrict the set of guesses it can make on the initial path.

- *Query response volume:* The query response volume, or $\mathsf{vol}_i = |\mathbf{DB}(\mathsf{initpath}(q_i))|$ can be inferred from the leaf intersection pattern. This allows an attacker to extract frequency information about the initial path in the database.
- *Character equality between the queries:* From the leaf intersection pattern, the attacker can find out if two queries start with the same characters. This can be done as follows. Consider queries $q_1 =$"ell" and $q_2 =$"elp" on the suffix tree in Figure 1. The attacker learns that both queries visit N_3. This is only possible if the first two characters of the two queries are the same. Note that the number of shared characters is leaked because the length of the initial paths of the two queries is one more than the string on the path of N_3. We write $\mathsf{charEq}(i, j) = k$ to mean that q_i and q_j have k common characters. In the example above, $\mathsf{charEq}(1, 2) = 2$. Character equality between the queries can be used by the attacker to refine its guesses for the queries.

Leakage Extraction. The above leakage components can be derived directly from the transcript of an execution of the CS scheme. We show how this can be done below.

Extracting the Length of the Initial Path. Recall that for query q, the client sends

$$\mathsf{F}_{\mathsf{sk}_1}(q[1,1]), \ldots, \mathsf{F}_{\mathsf{sk}_1}(q[1,|q|])$$

to the server. The server finds the largest m such that $\mathsf{F}_{\mathsf{sk}_1}(q[1,m])$ is in the encrypted database. Since m is equal to the length of the initial path by construction, the server can learn the length of the initial path from the transcript.

Extracting the Query Response Volume. In the last step of the search, suppose that the longest initial path matching the query is N. Then, the client will retrieve

$$\mathsf{L}[\mathsf{leafpos}(\mathsf{N})], \ldots, \mathsf{L}[\mathsf{leafpos}(\mathsf{N}) + \mathsf{num}(\mathsf{N}) - 1].$$

A permuted version of the above is used in the final scheme. The number of leaf nodes retrieved is equal to $\mathsf{num}(\mathsf{N})$, i.e., the the number of indices matching N. This allows the server to learn the query response volume directly.

Extracting Character Equality across Queries. For simplicity, consider queries q_i and q_j. For q_i, the client computes and sends to the server the PRF values $\mathsf{F}_{\mathsf{sk}_1}(q_1[1,1]), \ldots, \mathsf{F}_{\mathsf{sk}_1}(q_1[1,|q|])$. For q_2, these values are $\mathsf{F}_{\mathsf{sk}_1}(q_j[1,1]), \ldots, \mathsf{F}_{\mathsf{sk}_1}(q_j[1,|q|])$. Crucially, if q_i and q_j have a non-empty common prefix, i.e., there exists $k \geq 1$ such that $q_i[1,k] = q_j[1,k]$, then the first k PRF values from the two queries will be the same. As a result, the server can learn the character equality between the queries just by comparing the PRF values sent by the client.

Leakage Representation. In our attack, we process the leakage described above and represent the encrypted queries as lists of tokens where each token is an integer. If the same token appears in two encrypted queries, it means the corresponding characters in the two queries are the same. For example, for $q_1 =$"ell" and $q_2 =$"elp", since $\mathsf{charEq}(1, 2) = 2$, we can represent the encrypted version of the queries as $(1, 2, 3)$ and $(1, 2, 4)$ respectively. Our attack then tries to map the tokens to the alphabets used in the queries. It is worth noting that the encrypted queries we are interested in are only as long as the initial paths of the

original queries. This is because the server can only learn information about the initial path by design. We use the following notations in the description of our attack below. We abuse the notation and write $eq_i = (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})$ where m_i is the length of the initial path of query q_i. We write $|\mathbf{DB}(eq_i))|$ to mean the query response volume of (the initial path of) query q_i. Using the notation above, the leakage input to our attack is simply the collection of sequences of tokens $(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}$ where l is the number of queries made, and their corresponding query response volumes $(\mathsf{vol}_i)_{i=1}^{l}$ where $\mathsf{vol}_i = |\mathbf{DB}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))|$.

Our Attack. Given the leakage above, we design an inference attack that tries to recover the alphabets corresponding to the tokens. By doing so, we recover the initial path of the queries. There are three core components in our attack. First of all, we need to identify the set of guesses we want to make on each encrypted query. We call this the *candidate set* of the encrypted query. On a high level, given some auxiliary information, we can use the length of the initial path and the query response volume to filter out unlikely guesses for each encrypted query. Then, we can use the character equality leakage between the queries to reduce the size of the candidate sets further.

Secondly, once we obtain the candidate sets for all the encrypted queries, we need to have a statistical model to measure how well guesses on the encrypted queries fit the observed leakage. For this, we model the number of occurrences of the substrings with a Poisson distribution, where the parameters of the distribution are determined by some auxiliary data. Then, we compute the *likelihood* of a guess given the observed leakage components.

Finally, we need to use an efficient algorithm to search over the set of possible guesses and find the most likely one given the statistical model.

Identifying the Candidate Sets. Define $\mathsf{CandSet} : \mathbb{N}* \to \mathcal{P}(\Sigma^*)$ as a map that maps a sequence of tokens to a subset of all possible strings. Our goal is to find candidate sets of each encrypted query. We aim to make the candidate sets as small as possible so that the search procedure later will be as efficient as possible.

In our attack, we identify the candidate sets in two steps. In the first step, we rely on frequency analysis. Suppose that there is some auxiliary information $\mathsf{Aux} : \Sigma^* \to \mathbb{R}$ such that $\mathsf{Aux}(s)$ tells the attacker the expected query response volume of query s. The attacker can simply set $\mathsf{CandSet}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))\{s : |s| = m_i \wedge ||\mathbf{DB}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))| - \mathsf{Aux}(s)| < \sigma \cdot \sqrt{|\mathbf{DB}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))|}\}$ for some threshold σ. This is exactly what we do in our attack. Looking ahead, we describe in Section 3.3 how the threshold σ is determined in our experiments over real-world datasets.

However, the problem with the step above is that the candidate sets produced are often very large (on the order of 10^3 in size or more). For l queries, the search space is then roughly 10^{3l} in size. This makes efficient search over the space infeasible. As a result, we introduce a procedure to trim the search space before running the main algorithm of our attack. The idea of the trimming algorithm is that we can look at the candidate sets for each token (instead of the whole token sequence) by making use of the candidate sets and determine the most likely characters for each token. This can then be used to trim the candidate sets.

As an example, consider the following candidate sets. For token sequence $(1, 2, 3)$, we

have $\mathsf{CandSet}((1,2,3)) = \{\text{“ell”, “ali”}\}$. For token sequence $(1,2,4)$, $\mathsf{CandSet}((1,2,4)) = \{\text{“elp”, “bob”}\}$. From these, we can conclude that the character corresponding to token 1 is most likely “e” since it appears in both candidate sets. This then allows us to trim both of the candidate sets to $\mathsf{CandSet}((1,2,3)) = \{\text{“ell”}\}$ and $\mathsf{CandSet}((1,2,4)) = \{\text{“elp”}\}$ respectively. One complication here is that it is possible that the true query may not be included in the candidate set (because the observed query response volume is too far from the expected frequency of the substring). For this reason, we opt for a soft decision procedure where characters that do not appear in all candidate sets for a particular token may still be regarded as likely. In the paragraph below, we describe more formally how the trimming process is done.

We initialise an array $Arr : \mathbb{N} \times \Sigma \to \mathbb{N}$ that maps tokens and characters in the alphabet to an integer. This array is initialised with all possible token-character pairs and the values are set to zero. Then, for every token sequence $(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})$ we observe, we tally the characters that appear at each position in the candidate set $\mathsf{CandSet}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))$. For every token tk in the token sequence and every character c that appears in the corresponding positions, we increment $Arr(\mathsf{tk}, c)$ by 1. After this process is completed for all token sequences, we create a map $\mathsf{TokenCandSet} : \mathbb{N} \to \mathcal{P}(\Sigma)$ that maps each token to a subset of the alphabet. The entries of $\mathsf{TokenCandSet}$ are such that for token tk, $\mathsf{TokenCandSet}(\mathsf{tk})$ is set to the collection of characters c such that the values $Arr(\mathsf{tk}, c)$ are the t largest among $Arr(\mathsf{tk}, \cdot)$. If more than t characters share the largest value, then all characters with the largest value in $Arr(\mathsf{tk}, \cdot)$ are added to $\mathsf{TokenCandSet}(\mathsf{tk})$. The map $\mathsf{TokenCandSet}$ can then be used to trim $\mathsf{CandSet}$. This process is repeated until $\mathsf{CandSet}$ does not change by the trimming process. The pseudocode of the trimming process can be found in Algorithm 1 in Appendix D.1.

Statistical Model of the Observed Leakage. Given the candidate set $\mathsf{CandSet}$, the attacker can start making guesses $\mathsf{guess} : \mathbb{N}^* \to \Sigma^*$ on the plaintexts of the initial paths of the queries. However, we still need to measure how good each guess is so that we can pick the best one as the output of the attack. To do this, we model the query response volumes of the queries using Poisson distributions. These individual distributions are put together to form a joint distribution over all queries. We then apply standard statistical techniques to turn the joint distribution into a likelihood function.

The Likelihood Function. Due to space limitations, we present the derivation of the likelihood function in Appendix E.1.

In the rest of this section, we describe how the attacker can search for a guess that maximizes the likelihood function.

Consistency in the Guesses. Note that in the description of our attack above, guesses are made on each query separately. This means it is possible to have $\mathsf{guess}((1,2,3)) =$ “elp” and $\mathsf{guess}((1,2,4)) =$ “bob” even though we know the two queries must have the same first two characters. In our attack, we choose to make guesses iteratively and keep track of the guesses we make on the individual tokens. A new guess on a token is allowed to overwrite an old guess. Concretely, for the example above, we will make the guess $\mathsf{guess}((1,2,3)) =$ “elp” first followed by $\mathsf{guess}((1,2,4)) =$ “bob”. Since the second guess overwrites the guesses for token 1 and 2, it changes $\mathsf{guess}((1,2,3))$ to “bop”. This approach allows us to maintain

consistency of token equality at the cost of potentially sub-optimal guesses for the queries. We believe that this is not a major issue as the sub-optimal guesses yield low likelihood scores and they are unlikely to be accepted by the optimization algorithm we describe subsequently.

Maximizing the Likelihood Function. We use simulated annealing [70] to maximize the likelihood function. The corresponding pseudocode is described in Algorithm 2 in Appendix D.1 (the score function **Score**() is precisely the likelihood function described mathematically above). The procedure involves five subroutines:

- **Initial_solution**($\mathsf{CandSet}$): The algorithm takes as input the candidate set $\mathsf{CandSet}$ and outputs a guess guess which will be used as the initial solution of simulated annealing.
- **Neighbour**($\mathsf{guess}, \mathsf{CandSet}$): The algorithm takes as input the current guess guess and the candidate set $\mathsf{CandSet}$, and output a new guess guess' that is close to guess.
- **Score**($\mathsf{guess}, (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}, (\mathsf{vol}_i)_{i=1}^{l}, \mathsf{Aux}$): The algorithm takes as input a guess guess, a list of tokens $(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}$, a list of observed query response volumes $(\mathsf{vol}_i)_{i=1}^{l}$, and an auxiliary distribution Aux, and output a score sc.
- **Cooling**(T): The algorithm takes as input a temperature T and outputs a new temperature T'.
- **Accept_prob**($\mathsf{sc}, \mathsf{sc}', \mathsf{T}$): The algorithm takes as input two scores sc and sc' and a temperature T, and outputs 0 or 1 depending on the inputs and some randomness.

The five subroutines are combined as follows in a full run of simulated annealing:

- First, $\mathsf{guess} \leftarrow$ **Initial_solution**($\mathsf{CandSet}$) is run to get an initial guess guess.
- Next, a temperature T is initialised.
- Subsequently, the following procedure is executed iteratively for a fixed number of iterations:
  - The cooling subroutine **Cooling**(T) is invoked to obtain a new temperature T'. A neighbour $\mathsf{guess}' \leftarrow$ **Neighbour**($\mathsf{guess}, \mathsf{CandSet}$) of the current guess is also computed.
  - Then, a score $\mathsf{sc}' \leftarrow$ **Score**($\mathsf{guess}', (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}$, $(\mathsf{vol}_i)_{i=1}^{l}, \mathsf{Aux}$) is computed on the guess. Assume that the old score
    $$\mathsf{sc} \leftarrow \textbf{Score}(\mathsf{guess}', (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}, (\mathsf{vol}_i)_{i=1}^{l}, \mathsf{Aux})$$
    is kept by simulated annealing, it can call **Accept_prob**($\mathsf{sc}, \mathsf{sc}', \mathsf{T}$) to get an output 0 or 1.
  - If the output is 0, then nothing happens, and simulated annealing proceeds to the next iteration.
  - If the output is 1, the old guess guess is overwritten by the new guess guess', and simulated annealing proceeds to the next iteration.

3.3 Experimental Evaluation

We present experimental validation of our proposed attack against the CS scheme. We use two real-world datasets – (Simple) English Wikipedia and the Enron Email Corpus. A description of these datasets can be found in Appendix F. We show that our attack achieves a high success rate with practical efficiency while scaling to large datasets. Our attack code is publicly available on Github.[4]

Datasets. We parse English Wikipedia into 200K Wikipedia pages. From there, we randomly pick 100K pages and use them as the auxiliary dataset. From the remaining pages, we randomly pick 25K, 50K, 75K and 100K pages as the target dataset.

We parse the Enron Email Corpus into 400K emails. From there, we randomly pick 200K emails and use them as the auxiliary dataset. From the remaining emails, we randomly pick 50K, 100K, 150K and 200K emails as the target dataset.

Queries. We make 1K, 5K, and 10K queries on each target dataset. Each query is picked by uniformly randomly choosing a Wikipedia page/email, picking a random starting position in the text, and set a uniformly random query length between 3 and 15 (inclusive). We ensure that the queries are unique and the queries only contain English alphabetic characters, spaces, and hyphens.

Remark. We process the datasets in exactly the same way in the other experimental evaluations we present in this paper. Thus, details are omitted in the respective sections.

Attack Parameters. For English Wikipedia, we use $\sigma = 8$ and $\mathsf{t} = 12$ to build the candidate sets. For the Enron Email Corpus, we use $\sigma = 12$ and $\mathsf{t} = 14$ to build the candidate sets. These choices leads to the highest proportions of the candidate sets containing the correct guess experimentally.

The number of iterations of simulated annealing is set to 40M, 45M, and 60M for 1K, 5K, and 10K queries respectively.

Attack Results. We present the attack results in Figure 2. We report three metrics, namely (i) the fraction of queries where their longest prefixes in the (encrypted) suffix tree are recovered correctly (this is labelled as "Prefix" in the plots), (ii) the fraction of queries where the whole queries are recovered correctly (this happens when the longest prefix of a query in the (encrypted) suffix tree is the query itself. This is labelled as "Query" in the plots), and (iii) the fraction of the characters of the queries that are recovered correctly (this is labelled as " Character" in the plots).

Attack accuracy grows with the number of documents in the encrypted database and the number of queries. At 100K documents and 10K queries, our attack is able to achieve 20% query recovery rate on English Wikipedia. Similarly, at 200K documents and 10K queries, our attack is able to achieve 20% query recovery rate on the Enron Email Corpus.

[4] https://github.com/substring-sse-attacks/substring-sse-attacks/

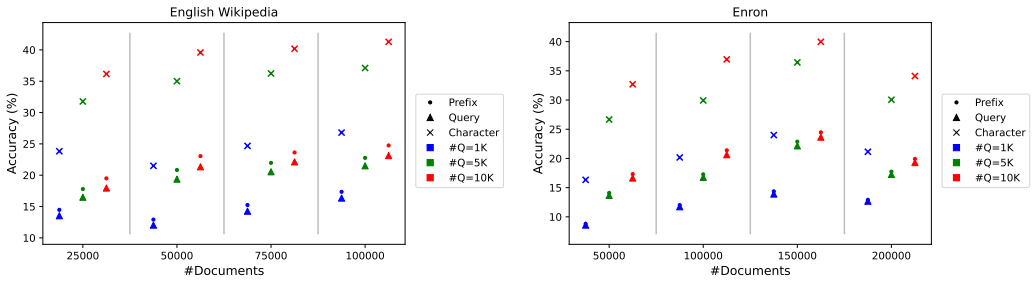

Figure 2: Attack results for the CS scheme.

More interestingly, the character recovery rate of our attack is significantly higher than the query recovery rate, at about 40% on English Wikipedia and 35% on the Enron Email Corpus in the same setting as above. This indicates that even in cases where the attacker cannot guess the whole query correctly, it has reliable guesses for the first few characters of the query.

While the query recovery rate may look relatively low compared to some other leakage-abuse attacks in the SSE literature, we stress that our attack setting is much harder. In particular, we put no restriction on the query response volume of the queries unlike the typical attacks against SSE schemes [3, 7, 16, 32, 33, 38, 58–60, 64]. In fact, for our attack on the Enron Email Corpus with 200K documents and 10K queries, about 30% of the queries have query response volume below 100. However, this does not prevent us from learning partial information about these queries due to the query prefix pattern.

4 Query Reconstruction Attack against the FJK+ Scheme

In this section, we describe our query reconstruction attack against the FJK+ substring-SSE scheme proposed by Faber *et al.* in [20].

4.1 The FJK+ Scheme

The Plaintext Scheme. We begin with a description of the scheme if it were a plaintext scheme. The plaintext scheme treats the documents as a collection of k-grams with their locations, i.e., $\mathbf{DB} = \{(\mathsf{kg}_i, \mathsf{ind}_i, \mathsf{pos}_i)\}$. We build two data structures to support substring queries. The first data structure is an inverted index where the keys are the k-grams, and the values are the list of matching indices and positions of the k-grams. The second data structure is a set, which contains all elements of **DB**. These two data structures are stored on the server.

To make a query on a substring, the substring is first parsed as a leading k-gram (known as the search term, or s-term), and a set of other k-grams (known as the cross terms, or x-terms)

with their offsets. For example, a query $q = \text{`}hello'$ may be parsed as $(\text{`}hel', (2, \text{`}llo'))$. One should pick the leading k-gram to be the k-gram with the smallest query response volume among the possible choices to minimize the computation complexity on the server (this is also recommended for minimizing the leakage).

To run the query on the server, the server first searches the first data structure with the leading k-gram. As a result, it obtains a list of document identifiers and positions within those documents $\{(\mathsf{ind}_i, \mathsf{pos}_i)\}$. Then, for every other k-gram with its offset, say (Δ, kg), the server can check if $(\mathsf{kg}, \mathsf{ind}_i, \mathsf{pos}_i + \Delta)$ is in the second data structure. If this holds for a particular i and for all k-grams and their offsets in the query, the server knows $(\mathsf{ind}_i, \mathsf{pos}_i)$ is a match.

The Encrypted Scheme. Now we turn our attention to turning the plaintext scheme described above into an encrypted scheme. The core idea is to use a PRF F which takes as input a PRF key $(\mathsf{sk}_X, \mathsf{sk}_I)$ and input of the form $(\mathsf{kg}_i, \mathsf{ind}_i, \mathsf{pos}_i)$. We use this PRF to generate the tags for $(\mathsf{kg}_i, \mathsf{ind}_i, \mathsf{pos}_i) \in \mathbf{DB}$ (known as the cross tags, or xtags) before inserting them into the second data structure (known as the cross set, or XSet).

PRF F has a special structure as follows:

$$\mathsf{F}((\mathsf{sk}_X, \mathsf{sk}_I), (\mathsf{kg}, \mathsf{ind}, \mathsf{pos})) = g^{f(\mathsf{sk}_X, \mathsf{kg}) f(\mathsf{sk}_I, \mathsf{ind})^{\mathsf{pos}}},$$

where g is a generator for some group G, and f is another PRF. This allows the client to modify the first data structure and issue encrypted substring queries.

Specifically, the first data structure will store the following entries instead. Suppose the matching locations for k-gram kg are $\{(\mathsf{ind}_i, \mathsf{pos}_i)\}$. We begin by creating an empty list in the data structure that is indexed by kg. Then, for each matching location $(\mathsf{ind}_i, \mathsf{pos}_i)$, we append to the list the following triple:

- A ciphertext e which encrypts ind_i and the encryption key for the ind_i-th document.
- $y \leftarrow f(\mathsf{sk}_I, \mathsf{ind}_i) \cdot z^{-1}$. The mask z depends on i.
- $v \leftarrow f(\mathsf{sk}_I, \mathsf{ind}_i)^{\mathsf{pos}} \cdot u^{-1}$. The mask u depends on kg and i.

The keys of the data structure are then masked using a PRF, so queries on the data structure do not leak the queries directly.

Suppose now that the client wants to make a query $(\mathsf{kg}_1, (\Delta_2, \mathsf{kg}_2))$. It will first make a query on kg_1, and the server will start working with the list of triples described above. For simplicity, let us assume that there is only one match for kg_1 and it is $(e, f(\mathsf{sk}_I, \mathsf{ind}_i) \cdot z^{-1}, f(\mathsf{sk}_I, \mathsf{ind}_i)^{\mathsf{pos}} \cdot u^{-1})$. The goal of the client is to facilitate the server in computing $t \leftarrow \mathsf{F}((\mathsf{sk}_X, \mathsf{sk}_I), (\mathsf{kg}_2, \mathsf{ind}, \mathsf{pos} + \Delta_2))$. If t is in the XSet, then the server learns that there is a match for the whole query, and it should return e to the client (after which the client can decrypt e and retrieve the document from the information encrypted in e).

The client simply sends Δ_2 and $\mathsf{xtk} \leftarrow g^{f(\mathsf{sk}_X, \mathsf{kg}_2) \cdot z^{\Delta_2} \cdot u}$ to the server. The server then

computes $\mathsf{F}((\mathsf{sk}_X, \mathsf{sk}_I), (\mathsf{kg}, \mathsf{ind}, \mathsf{pos}))$ by combining xtk with y and v as follows:

$$\begin{aligned}\mathsf{xtk}^{y^{\Delta_2} \cdot v} &= g^{f(\mathsf{sk}_X, \mathsf{kg}_2) \cdot z^{\Delta_2} \cdot u \cdot f(\mathsf{sk}_I, \mathsf{ind}_i)^{\Delta_2} \cdot z^{-\Delta_2} \cdot f(\mathsf{sk}_I, \mathsf{ind}_i)^{\mathsf{pos}} \cdot u^{-1}} \\ &= g^{f(\mathsf{sk}_X, \mathsf{kg}_2) \cdot f(\mathsf{sk}_I, \mathsf{ind}_i)^{\mathsf{pos}+\Delta_2}}.\end{aligned}$$

More generally, if the query contains more x-terms, the client has to send the offsets and x-tokens associated with those x-terms to the server as well; if there is more than one matching location for the s-term, the client and the server need to repeat the process above for all matching locations.

4.2 Our Attack Against the FJK+ Scheme

We now describe our query reconstruction attack against the FJK+ scheme. We begin by describing the leakage components we used in our attack and show that they can be extracted from the transcript of the scheme itself. We then give a description of our attack.

Leakage Components. For completeness, we give a full description of the leakage profile of the FJK+ scheme in Appendix C.2. We focus here on the leakage components that we use in our attack. Given queries $\{q_i\}$, such that $q_i = (\mathsf{kg}_i, (\Delta_{i,1}, \mathsf{kg}_{i,1}), \ldots, (\Delta_{i,l_i}, \mathsf{kg}_{i,l_i}))$, our attack use the following leakage components: (1) N, the total number of k-grams in all documents, (2) s-term equality pattern, (3) x-term equality pattern when it appears (we will give more details later), (4) the number of matches for each of the s-terms, (5) the number of matches for each $(\mathsf{kg}_i, (\Delta_{i,j}, \mathsf{kg}_{i,j}))$, and (6) the constraints between different s-terms. We demonstrate how the leakage components are represented in our attack below.

Query Labels. We label the i-th query as

$$\mathsf{lk}_i = (\mathsf{sid}_i, (\Delta_{i,1}, \mathsf{xid}_{i,1}), \ldots, (\Delta_{i,l_i}, \mathsf{xid}_{i,l_i})),$$

such that (i) for any i, j, $\mathsf{sid}_i = \mathsf{sid}_j$ if $\mathsf{kg}_i = \mathsf{kg}_j$, i.e., the i-th and the j-th queries share the same s-term, and (ii) for any i, j, k, l such that $\mathsf{kg}_i \neq \mathsf{kg}_k$, $\mathsf{xid}_{i,j} = \mathsf{xid}_{k,l}$ if there exists a location in a document $(\mathsf{ind}, \mathsf{pos})$ such that $(\mathsf{kg}_i, \mathsf{ind}, \mathsf{pos} - \Delta_{i,j}) \in \mathbf{DB}$, $(\mathsf{kg}_k, \mathsf{ind}, \mathsf{pos} - \Delta_{k,l}) \in \mathbf{DB}$, $\mathsf{kg}_{i,j} = \mathsf{kg}_{k,l}$, and $(\mathsf{kg}_{i,j}, \mathsf{ind}, \mathsf{pos}) \in \mathbf{DB}$.

Volume Maps. In addition to the labels above, we define two maps $\mathsf{vol}_s(\cdot)$ and $\mathsf{vol}_x(\cdot, \cdot, \cdot)$. Here, vol_s maps the s-term identifier we assign to the s-term kg_i to the number of matches of kg_i in the database. vol_x maps the s-term identifier we assign to the s-term kg_i, an offset $\Delta_{i,j}$, and the x-term identifier we assign to $\mathsf{kg}_{i,j}$, to the number of matches for $(\mathsf{kg}_i, (\Delta_{i,j}, \mathsf{kg}_{i,j}))$ in the database.

s-term Intersection Pattern. The s-term intersection pattern SIP is a set which contains $(\mathsf{sid}_i, \mathsf{sid}_k, \Delta)$ if $\mathsf{kg}_i \neq \mathsf{kg}_k$, and there exists some $j, l, \mathsf{ind}, \mathsf{pos}$ such that $(\mathsf{kg}_i, \mathsf{ind}, \mathsf{pos} - \Delta_{i,j}) \in \mathbf{DB}$, $(\mathsf{kg}_k, \mathsf{ind}, \mathsf{pos} - \Delta_{i,j} - \Delta) \in \mathbf{DB}$, $\mathsf{kg}_{i,j} = \mathsf{kg}_{k,l}$, and $(\mathsf{kg}_{i,j}, \mathsf{ind}, \mathsf{pos}) \in \mathbf{DB}$.

Concrete Example. Consider the following queries:

- $q_1 = (\mathsf{kg}_1, (\Delta_2, \mathsf{kg}_2)) = (\text{'mot'}, (3, \text{'her'}))$,
- $q_2 = (\mathsf{kg}_3, (\Delta_4, \mathsf{kg}_4)) = (\text{'mot'}, (3, \text{'ion'}))$,
- $q_3 = (\mathsf{kg}_5, (\Delta_6, \mathsf{kg}_6), (\Delta_7, \mathsf{kg}_7)) = (\text{'oth'}, (2, \text{'her'}), (5, \text{'wis'}))$.

Omitting N, the leakage representation would look as follows:

- $\mathsf{lk}_1 = (1, (3, 1)), \mathsf{lk}_2 = (1, (3, 2)), \mathsf{lk}_3 = (2, (2, 1), (5, 3))$.
- $\mathsf{vol}_s(1)$ is the s-term support size of 'mot', $\mathsf{vol}_x(1, 3, 1)$ is the results pattern of 'mother', and so on.
- $\mathsf{SIP} = \{(1, 2, 1)\}$, meaning that the second and the third characters of the s-term labelled by 1 ('mot' in q_1 and q_2) are the same as the first and the second characters of the s-term labelled by 2 ('oth' in q_3).

Leakage Extraction. We argue that the leakage representation we described in Section 4.2 can be extracted from an actual semi-honest server, and we are not exploiting any of the over-specified components of the leakage profile described in Appendix C.2.

Firstly, the server can learn N from the first encrypted data structure described in Section 4.1. It can also tell when the s-terms between two queries are the same as the same query token is used for this part of the query. The server can tell when the x-terms are the same when the x-tag computed by the server matches one from before. Of course, the server learns the offsets $\Delta_{i,j}$ as they are sent to the server explicitly. Secondly, the server learns vol_s as it is explicitly leaked to the server. The server learns vol_x as the server is explicitly checking for matches for the s-term and each of the x-term and the corresponding offset in each query. Finally, the server learns SIP by looking for repeated x-tags; it can infer the offsets between the s-terms by checking the offsets of the x-terms.

Our Attack. We assume that the attacker has some auxiliary information about the database $(\mathsf{Aux}_s, \mathsf{Aux}_x)$. Here Aux_s maps a k-gram to the expected frequency of the k-gram in the text corpus and Aux_x maps a k-gram kg_i (treated as the s-term), an offset Δ, and a second k-gram kg_j (treated as the x-term) to the expected frequency of the query $(\mathsf{kg}_i, (\Delta, \mathsf{kg}_j))$ in the text corpus. The goal of the attacker is to produce guesses guess_s and guess_x, such that guess_s maps the s-term identifiers (sid_i) to k-grams, and guess_x maps the x-term identifiers ($\mathsf{xid}_{i,j}$) to k-grams. As the offsets ($\Delta_{i,j}$) are learnt by the server explicitly, correct guesses guess_s and guess_x allow the server to reconstruct the queries entirely.

Attack Overview. Our attack can be broken down into three steps. Detailed pseudocode for these steps are shown in Algorithms 3, 4, and 5 in Appendix D.2 respectively.

In the first step (Algorithm 3), we make use of the s-term support size leakage (captured in vol_s) and the auxiliary information on it (Aux_s) to identify the candidate sets for each s-term in the query. We keep the candidate sets in a map $\mathsf{CandSet}_s$ which maps each sid_i to a set of k-grams.

We then make use of the s-term intersection pattern SIP to refine the s-term candidates. For example, if $(1, 2, 1) \in \mathsf{SIP}$, for every $\mathsf{kg} \in \mathsf{CandSet}_s(1)$, we check if there is $\mathsf{kg}' \in \mathsf{CandSet}_s(2)$

such that $\mathsf{kg}[2:] = \mathsf{kg}'[:3]$. If there is no such kg', we remove kg from $\mathsf{CandSet}_s(1)$. We repeat this with $(2, 1, -1)$ to prune the candidate set for sid_2. We go back and forth with $(1, 2, 1)$ and $(2, 1, -1)$ until the candidate sets for sid_1 and sid_2 do not change any more. This process is repeated for all entries in SIP.

At this stage, we are ready to make guesses on the s-term identifiers and the x-term identifiers. Similar to our attack on the CS scheme (Section 3.2), our attack is built using simulated annealing. The key difference between our attack on the CS scheme and $\mathsf{FJK+}$ scheme is that now we run two rounds of simulated annealing. We give more details below.

First Round of Simulated Annealing. In the first round (Algorithm 4), we design an iteration of simulated annealing as follows. Given the leakage representation of the queries, say $\{\mathsf{lk}_1, \ldots, \mathsf{lk}_n\}$, we construct a map L that maps the s-term identifiers to a set of query leakages. That is, for a given sid, $L(\mathsf{sid})$ contains all lk_i's such that the s-term identifier of lk_i is sid. Using the example we introduced in Section 4.2, we have $L(1) = \{\mathsf{lk}_1, \mathsf{lk}_2\}$ and $L(2) = \{\mathsf{lk}_3\}$.

An iteration of simulated annealing is broken down into steps. In each step, the attack focuses on all queries with a particular s-term identifier (i.e., $L(\mathsf{sid})$). The rationale behind this is that the contribution of the queries in $L(\mathsf{sid})$ to the likelihood function of an assignment of the s-terms and x-terms (which we try to maximize; see Appendix E.2 for a full description) is independent from the contribution of other queries to the likelihood, unless the queries with different s-terms share the same x-term (and these x-terms need to be identified as the same x-term in the conditional intersection pattern).

In each step, the attack makes a new guess on the s-term, and new guesses on *all* of the x-terms in *all* of the queries in $L(\mathsf{sid})$ based on the auxiliary information $(\mathsf{Aux}_s, \mathsf{Aux}_x)$. We compute a new likelihood score based on these new guesses. If the new guesses are accepted, we update guess_s and guess_x. The steps are made in decreasing s-term support sizes, i.e., if $\mathsf{vol}_s(\mathsf{sid}) > \mathsf{vol}_s(\mathsf{sid}')$, we run a step on $L(\mathsf{sid})$ before the step on $L(\mathsf{sid}')$.

In more detail, the new guess made on the s-term is based on $\mathsf{CandSet}_s$ produced in the second step of the attack. For the guesses on the x-terms, we opt to not build candidate sets, as the plausible guesses for the x-terms depend on the s-term of the query. So we construct the candidate set for each x-term on the fly in our attack. Then, we pick a random candidate in the candidate set as our guess for the x-term.

One caveat is that we do not make a new guess on an x-term each time we see it. To see what we mean, consider the example in Section 4.2. Suppose the first step of our attack in a round visits all queries with $\mathsf{sid} = 1$, and the second step visits all queries with $\mathsf{sid} = 2$. This means the attack visits $\{\mathsf{lk}_1, \mathsf{lk}_2\}$ in the first step, and $\{\mathsf{lk}_3\}$ in the second step. Note that $\mathsf{xid} = 1$ appears in both lk_1 and lk_3. So if we make a new guess on $\mathsf{xid} = 3$ every time we see it, we will make a guess on $\mathsf{xid} = 1$ in lk_1, and override the guess in lk_3. This is a bad idea as we traverse through the query leakages in decreasing order of their s-term support sizes. As we move on in a round, the volume leakage $\mathsf{vol}_x(\cdot, \cdot)$ will become increasingly small, and the quality of the candidates for the x-term will be worse. This leads to worse guesses for the x-term. For this reason, we only make one guess for each x-term (when we see it for the first time).

However, there is an exception to the rule above: if the offset Δ when the guess on an x-term is made is larger in absolute value than the offset Δ' seen in the current query (i.e., $|\Delta| > |\Delta'|$), we allow for a new guess on the x-term. The decision is based on our observation that the guesses on the x-terms are more accurate when the s-term and the x-term are close to each other. This exception allows us to improve the accuracy of the attack significantly.

In the pseudocode, we omit the description of **Cooling**() and **Accept_prob()**, as they are the same as those in Algorithm 2. We give a description of the likelihood function in Appendix E.2.

Second Round of Simulated Annealing. After the first round of simulated annealing, we will have several correct guesses for the s-terms and the x-terms. That means we are ready to improve the guesses for the s-terms and x-terms individually. That is precisely what we do in the second round of simulated annealing (Algorithm 5). An iteration of this second round works in a similar way to that of the first round. We begin by sorting the queries by the support size of their s-terms. Then, we make new guesses on the s-term and x-terms one by one, and accept the new guess if it improves the likelihood score. The same caveat as in the first round of simulated annealing applies here. That is, we make a new guess on an x-term only if (1) we have not made a guess on the x-term before, or (2) the previous guess was made with an s-term further away than the current one. In the pseudocode, we omit the description of **Cooling**() and **Accept_prob()** as they are the same as those in Algorithm 2.

4.3 Experimental Evaluation

Attack Parameters. We use $\mathsf{t}_s = (0.5, 2)$ to trim the s-term candidate sets with $\mathsf{vol}_s(\mathsf{sid}) <$ 12K, and $\mathsf{t}_s = (0.7, 1.3)$ otherwise. We use $\mathsf{t}_x = (0.5, 1.5)$ to trim the x-term candidate sets. These choices result in s-term and x-term candidate sets containing the correct guesses most of the time in our experiments.

Attack Results. We present the attack results in Figure 3. We report five metrics, namely: (i) the fraction of the unique s-terms that the attacker has guessed correctly (even if the same s-term appears in two queries, we only count it once) – this is labelled as "Uniq s-term" in the plots, (ii) The fraction of the s-terms that the attacker has correctly across the queries (the s-terms are counted with repetition) – this is labelled as "s-term" in the plots, (iii) the fraction of the unique x-terms that we have guessed correctly – this is labelled as "Uniq x-term" in the plots, (iv) the fraction of the x-terms that the attacker has guessed correctly across the queries – this is labelled as "x-term" in the plots, and (v) the fraction of the queries that the attacker has guessed correctly – this is labelled as "Query" in the plots.

We make two important observations. Firstly, the recovery rates of the s-terms and x-terms with repetition are significantly higher than the unique counterparts. This indicates that a small fraction of s-terms and x-terms appear across many queries, making their recovery much easier. The root cause of this problem is that the $\mathsf{FJK}+$ scheme operates with n-grams, which is a significant departure from the original OXT scheme [9] on which the $\mathsf{FJK}+$ scheme

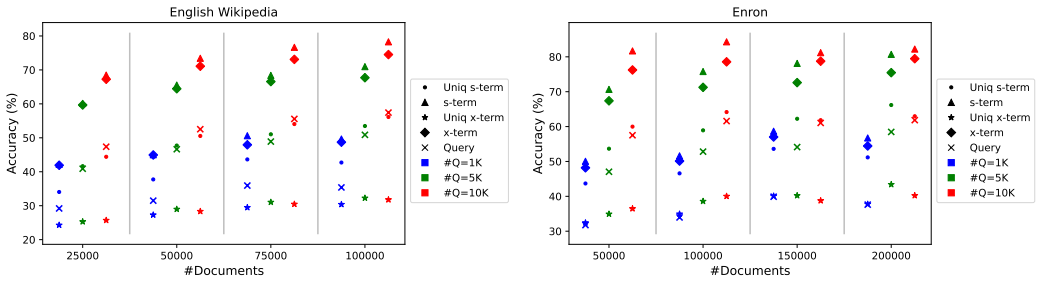

Figure 3: Attack results for the FJK+ scheme.

is based. As a result, the original intuition for the security of the OXT scheme (that the s-term has very low frequency, and the conditional intersection pattern is small) no longer holds.

Secondly, the query recovery rate grows significantly with the number of queries. This is because the conditional intersection pattern "grows" with the number of queries, so there are more and more restrictions on the s-terms of the queries.

5 Query Reconstruction Attack against the HLK Scheme

In this section, we describe our query reconstruction attack against the HLK substring-SSE scheme proposed by Hahn *et al.* in [36].

5.1 The HLK Scheme

Setup. The HLK scheme operates with k-grams, like the FJK+ scheme. For simplicity, we give a running example with a single string. The scheme generalises naturally to multiple strings. Consider the string $S =$ "*banana*". In the setup phase, the client begins by tokenise the string. The tokenisation produces k-grams and the positions for which they appear in. Assuming $k = 3$, the client obtains

$$(\text{“}ban\text{”}, 1), (\text{“}ana\text{”}, 2), (\text{“}nan\text{”}, 3), (\text{“}ana\text{”}, 4), (\text{“}na\_\text{”}, 5), (\text{“}a\_\_\text{”}, 6)$$

after tokenisation.

Then, the client sorts the tokens by the k-grams lexicographically. That is, the list of tokens becomes $(\text{“}a\_\_\text{”}, 6), (\text{“}ana\text{”}, 2), (\text{“}ana\text{”}, 4), (\text{“}ban\text{”}, 1), (\text{“}na\_\text{”}, 5), (\text{“}nan\text{”}, 3)$. After that, the client encrypts the positions of the tokens in the order of the list, and uploads it to the server. Concretely, the encrypted list T will be $T[0] = \mathsf{Enc}_{\mathsf{sk}}(6)$, $T[1] = \mathsf{Enc}_{\mathsf{sk}}(2), \ldots, T[6] = \mathsf{Enc}_{\mathsf{sk}}(3)$, for some encryption algorithm Enc with an encryption key sk.

Finally, the client stores the starting and ending indices for each k-gram in the sorted list

as its secret state. Concretely, the secret state ST of the client will be $\mathsf{ST}[\text{“}a\_\_\text{”}] = (1,1)$, $\mathsf{ST}[\text{“}ana\text{”}] = (2,3), \ldots, \mathsf{ST}[\text{“}nan\text{”}] = (6,6)$.

Query. Given a query string, say $q =$ “*bana*”, the client begins by tokenising the query. In the example, q is tokenised into (“*ban*”, 0), (“*ana*”, 1). In general, the k-grams produced by the tokenisation process do not need to overlap, except for the penultimate and last k-grams. Given the tokens, the client looks up the k-grams in its secret state, and finds the starting and ending indices of the encrypted matching positions stored on the server. In the example, the starting and ending indices for “*ban*” are $(4,4)$, and that for “*ana*” are $(2,3)$. The client retrieves $T[2], T[2]$, and $T[4]$ from the server and decrypts them. Finally, the client checks if the matching positions for the k-grams produce matching positions for the whole query. For example, the client learns that “*ban*” appears at position 1, so it checks if “*ana*” appears in position 2. That indeed happens, so the substring starting from position 1 is a match for q. On the other hand, “*ana*” in position 4 does not yield a match.

The original paper does not specify the tokenisation procedure for the queries. We assume a simple tokenization strategy where a query is tokenised as $q = ((\mathsf{kg}_1, \Delta_1), \ldots, (\mathsf{kg}_l, \Delta_l))$, where $\Delta_1 = 0, \Delta_2 = k, \ldots, \Delta_{l-1} = k(l-1)$. In other words, a query is tokenised as non-overlapping k-grams except for the last k-gram (which may overlap with the second last k-gram).

The paper also does not specify whether the matching positions are retrieved in the order that the k-grams appear in the query. If they do, the HLK scheme will have more leakage. In our attack, we assume the matching positions are retrieved in the increasing order of the indices in the encrypted database, so the order of the k-grams in the query is not known to the server.

Substring Queries on Multiple Strings. The scheme described above can be easily converted into one that supports substring queries on multiple strings (or documents). One simply tokenises the strings one by one, and store the string indices and the positions instead of just the positions in the first step. As an example, instead of (“*ban*”, 1) indicating that “*ban*” appears at position 1, the tokenisation will produce (“*ban*”, (1, 1)) to indicate that “*ban*” appears at position 1 in the first string.

5.2 Our Attack Against the HLK Scheme

Leakage Profile of the HLK Scheme The original paper does not provide a description of the leakage profile of the HLK scheme. However, the leakage profile is fairly easy to describe. For simplicity, we represent a query as a list of k-grams and offsets $q = ((\mathsf{kg}_1, \Delta_1), \ldots, (\mathsf{kg}_l, \Delta_l))$.

The leakage profile of the HLK scheme is a tuple (N, IP) where N is the total number of k-grams in all documents, and IP is the index pattern that maps each k-gram in a query to the starting and ending indices of the k-gram in the secret state of the client. This information can be constructed from the documents directly without any secret information. To continue our example, a query “*bana*” will allow the server to learn that the indices of the queried

k-grams are $(2, 3)$ and $(4, 4)$.

Representing Leakage. Given a query $q = ((\mathsf{kg}_1, \Delta_1), \ldots, (\mathsf{kg}_l, \Delta_l))$, we use

$$\{(\mathsf{kid}_1, \mathsf{idx}_{1,1}, \mathsf{idx}_{1,2}), \ldots, (\mathsf{kid}_{l'}, \mathsf{idx}_{l',1}, \mathsf{idx}_{l',2})\}$$

to represent the leakage from the query, where kid_i is an identifier we assign to the i-th k-gram in the query. If the same k-gram appears in two queries, the corresponding k-gram identifiers will be the same in the leakage representation. $\mathsf{idx}_{i,1}$ is the starting index of kg_i, and $\mathsf{idx}_{i,2}$ is the ending index of kg_i. The volume information for kg_i is given by $\mathsf{idx}_{i,2} - \mathsf{idx}_{i,1} + 1$. In general, it is possible for a query to contain repeated k-grams. In that case, the leakage will contain fewer items than the number of k-grams in the tokenised query.

Our Attack. We describe our attack with the HLK scheme instantiated with 3-grams for simplicity. We assume that the attacker has some auxiliary information about the database, as follows: (i) rank maps a 3-gram kg_i to its expected rank in the database (i.e., the expected value for $\mathsf{idx}_{i,1}$), (ii) vol maps a 3-gram kg_i to its expected frequency (i.e., the expected value for $\mathsf{idx}_{i,2} - \mathsf{idx}_{i,1}$),(iii) Aux_3 is a set containing elements of the form $(\mathsf{kg}_i, \mathsf{kg}_j)$. Here, $(\mathsf{kg}_i, \mathsf{kg}_j) \in \mathsf{Aux}_3$ if $\mathsf{kg}_i \| \mathsf{kg}_j$ is a common 6-gram in the auxiliary database, and (iv) $\mathsf{Aux}_{\leq 3}$ is a set containing elements of the form $(\mathsf{kg}_i, \mathsf{kg}_j)$. Here, $(\mathsf{kg}_i, \mathsf{kg}_j) \in \mathsf{Aux}_{\leq 3}$ if one of the following three conditions holds: (1) $\mathsf{kg}_i[2:] = \mathsf{kg}_j[1:3]$ and $\mathsf{kg}_i \| \mathsf{kg}_j[3]$ is a common 4-gram in the auxiliary database, (2) $\mathsf{kg}_i[3] = \mathsf{kg}_j[1]$ and $\mathsf{kg}_i \| \mathsf{kg}_j[2:]$ is a common 5-gram in the auxiliary database, or (3) $\mathsf{kg}_i \| \mathsf{kg}_j$ is a common 6-gram in the auxiliary database.

Our attack will use rank and vol to recover the k-grams from the leakage. The attack will then use Aux_3 to recover the order of all of the 3-grams except the last 3-gram. Finally, the attack will use $\mathsf{Aux}_{\leq 3}$ to recover the last 3-gram.

Attack Overview. Our attack is shown in detail in Algorithm 6 in Appendix D.3. It runs two subroutines repeatedly for a few rounds. The first subroutine (Algorithm 7 in Appendix D.3) is responsible for identifying the candidate sets for each k-gram identifier based on the auxiliary information rank and vol. In the first round, this is effectively naive frequency analysis. The size of the candidate sets are controlled by two parameters, namely δ and σ. Here, δ controls how far the rank of the candidate is allowed to be from the observed rank of the 3-gram, while σ controls how different the frequency of the candidate is allowed to be from the observed frequency of the 3-gram.

In the other rounds, the first subroutine receives the candidate sets trimmed by the second subroutine as one of its inputs. The goal of the first subroutine in these rounds is to identify good candidate sets and rebuild the rest. In more detail, in the first step of the subroutine, we gather the k-gram identifiers for which we are left with a unique candidate. These unique candidates are effectively the guesses we have for the k-gram identifiers. Recall that we know the relative rank between the k-gram identifiers, meaning that for k-gram identifiers kid_i and kid_j, we know if $\mathsf{idx}_{i,1} < \mathsf{idx}_{k_1}$ (or not). But then guesses we have for the k-gram identifiers must obey the same relation in terms of lexicographical ordering, i.e. $\mathsf{idx}_{i,1} < \mathsf{idx}_{k_1} \iff \mathsf{kid}_i < \mathsf{kid}_j$. We count how many times each k-gram identifier with a unique candidate violates this requirement, and we keep the top 90% of the k-gram

identifiers with the least numbers of violations. For the other k-gram identifiers, including the ones with a larger candidate set, we rerun the frequency analysis on them with a larger σ.

The second subroutine (Algorithm 8 in Appendix D.3) receives the candidate sets produced by the first subroutine as one of its inputs. The goal is to trim the candidate sets so that we can uniquely identify as many k-grams as possible. We do this by looking at the leakage query by query, in the order of short queries to long queries. For simplicity, consider a query that consists of two 3-grams $((\mathsf{kg}_1, 0), (\mathsf{kg}_2, 3))$ and suppose the attacker knows that the length of the query is 6. The attacker can represent the leakage from the two 3-grams as $\{(\mathsf{kid}_i, \mathsf{idx}_{i,1}, \mathsf{idx}_{i,2}), (\mathsf{kid}_j, \mathsf{idx}_{j,1}, \mathsf{idx}_{j,2})\}$, where kid_i, kid_j are the 3-gram identifiers the attacker assigned to the 3-grams and the other components are the pertinent indices. The fact that kid_i and kid_j appears together tells the attacker that either $\mathsf{kg}_1 \| \mathsf{kg}_2$ or $\mathsf{kg}_2 \| \mathsf{kg}_1$ is a common substring in the language. So it can simply remove unlikely candidates for kid_i and kid_j based on how likely their combinations are in the language.

In practice, the attacker does not know the exact length of the query, so instead of assuming kid_i and kid_j are for disjoint 3-grams, the attacker has to consider three cases: (1) kg_1 and kg_2 overlap in two positions, (2) kg_1 and kg_2 overlap in a single position, and (3) kg_1 and kg_2 have no overlap. All of these cases are included in the auxiliary information $\mathsf{Aux}_{\leq 3}$ so the attacker only has to perform a single check. Of course, this special case is only applicable to the last two k-grams. For the other k-grams, we can assume that they are disjoint, and use Aux_3 as the auxiliary information.

Finally, after running the two subroutines for a few rounds, we expect many of the candidate sets to contain a single candidate, which means we have a unique guess for those k-grams. However, we still need to recover the order of the k-grams in the query. To do so, we make use of the auxiliary information Aux_3 and $\mathsf{Aux}_{\leq 3}$ in the same way we did in the second subroutine. The pseudocode for this procedure is shown in Algorithm 9 in Appendix D.3.

Design Rationale. We know the real k-grams corresponding to the k-gram identifiers should obey the rank relations in the leakage. However, as our guesses can be false, using the rank relations as a hard constraint will remove many correct guesses. Therefore, we opt to use the rank relations as a soft constraint and only remove the guesses that violate the rank relations the most. Furthermore, most of the time, the reason why the rank relations are violated is because the unique candidates themselves are wrong, as the correct candidates were not included in the initial candidate set before running the second subroutine. By increasing σ between the rounds, we make it more and more likely to include the correct candidates in the candidate sets. On the other hand, we want to start with a small σ so that the initial candidate sets will be reasonably small, and so that the second subroutine allows the attacker to produce unique candidates by eliminating unlikely ones.

5.3 Experimental Evaluation

Attack Parameters. For English Wikipedia, we use $\delta = 0.004$ and $\sigma = 8$ as the attack parameters. The number of iterations in the attack ($\mathsf{iter}_{\mathsf{max}}$ in Algorithm D.3) is set to 2,

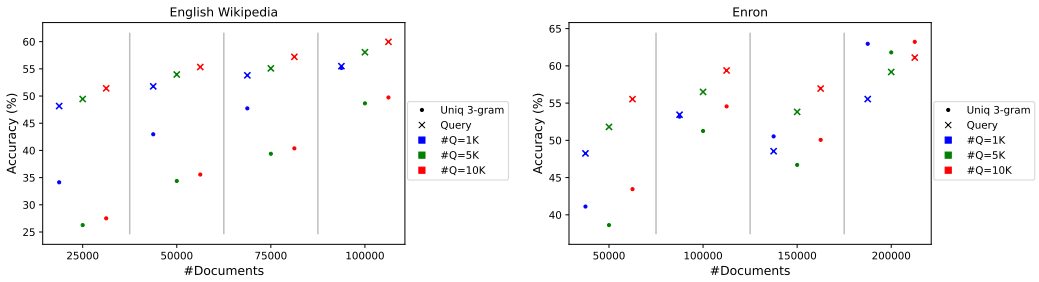

Figure 4: Attack results for the HLK scheme.

4, 5, and 7 for 25K, 50K, 75K, and 100K documents, respectively. For the Enron Email Corpus, we use $\delta = 0.008$ and $\sigma = 14$ as the attack parameters. The number of iterations is set to 15, 15, 30, and 30 for 50K, 100K, 150K, and 200K documents, respectively. These parameters produce the best query recovery rates in our experiments.

Attack Results. We present the attack results in Figure 4. We report two metrics, namely (i) the fraction of the unique 3-grams from the queries that the attacker has recovered correctly (even if the same 3-gram appears in two queries, we only count it once). This is labelled as "Uniq 3-gram" in the plots, and (ii) the fraction of the queries that the attacker has recovered correctly.

We make two important observations. Firstly, the unique 3-gram recovery rate of our attack is not particularly high. In fact, some of rates get lower as the number of queries increases (since more low-frequency 3-grams are queried as the number of queries increases). However, this does not stop the query recovery rate from increasing. It indicates that natural languages are well-structured at the 3-gram level, so that the bad guesses for the 3-grams can be easily eliminated. Secondly, recall that the HLK scheme does not leak the relative positions of the 3-grams from the queries. Despite this, our attack against the HLK scheme achieves similar query recovery rates to our attack against the FJK+ scheme. This indicates that hiding the relative positions of the n-grams by itself is not sufficient for building a secure n-gram-based substring-SSE.

6 Discussion

Our work initiates the study of leakage cryptanalysis of substring-SSE. We proposed novel query reconstruction attacks against the three most prominent, practically efficient, highly cited, and technically diverse substring-SSE schemes – the CS scheme [13], the FJK+ scheme [20], and the HLK scheme [36]. We implemented our attacks and experimentally validated their high query-recovery rate and practical efficiency over two real-world datasets – English Wikipedia and the Enron Email Corpus. Our experiments show that our attacks achieves high success rate with reasonable practical efficiency while scaling smoothly to large datasets, even without any restrictions on the query space (unlike known attacks against SSE for keyword search, where the query reconstruction rates are significantly lower with-

out such restrictions). For English Wikipedia (100K Wikipedia pages, 10K queries each of length between 3 and 15), the query reconstruction rates against CS, FJK+ and HLK are 23%, 57% and 60%, respectively. For the Enron Email Corpus (200K emails, 10K queries each of length between 3 and 15) the corresponding query reconstruction rates are 19%, 62% and 61%, respectively. For the attack against CS, we also achieve 41% and 34% character recovery rate for the two datasets, respectively.

Additional Substring-SSE Schemes. It would be interesting to cryptanalyse the leakage of other substring-SSE schemes. To the best of our knowledge, the three substring-schemes that we target in this paper (CS [13], FJK+ [20] and HLK [36]) cover essentially all fundamental techniques underlying the literature on practically efficient substring-SSE. In particular, the vast majority of other substring-SSE schemes (such as [52,69,71–73]) use storage/query-efficient variants of the same data structure and substring-search procedure as in the CS scheme. Consequently, all of these schemes have very similar leakage profiles to the CS scheme, and we believe that it should be possible to extend our attack on the CS scheme to these schemes. We leave this to future work. The authors of [57] proposed an SSE scheme for substring search that supports a slightly different substring search functionality; it is effectively an extension of standard SSE for single-keyword search that additionally allows querying substrings of keywords. We also leave it as an interesting open question to analyze the leakage of this scheme.

A few substring-SSE proposals [4, 56] use generic and computationally expensive cryptographic tools (such as ORAM, PIR, or FHE) to suppress leakage. Such schemes are unlikely to be vulnerable to our attacks. However, these schemes are also impractical at meaningful scale. As a concrete comparison, the authors of [20] report a query-processing time of 40s for the FJK+ scheme on a 10TB-sized database, while [4] reports a query-processing time of 40s for their FHE-based substring-SSE scheme on a single string of length 10 with 1080 characters. For real-world applications where performance is paramount, one would naturally prefer practically efficient substring-SSE schemes.

Countermeasures to Leakage Cryptanalysis. Recent works have investigated techniques for suppressing certain kinds of leakage (notably, response volume leakage) from SSE schemes for keyword search [24, 28, 34, 42, 62] and range queries [17]. Since our attacks exploit various forms of volume leakage (e.g., the response volume in the CS scheme, the size of the result pattern in FJK+, or the n-gram frequency leakage in HLK) a natural question to ask is whether one could generically apply volume leakage suppression to these schemes as a possible countermeasure against our attacks. Unfortunately, this does not work. For instance, in the case of the CS scheme, the volume leakage that we exploit in our attack is, in fact, crucial to the query execution methodology of the scheme itself (the leakage arises from a value stored in each node of the suffix tree that is used to determine to how the suffix tree is traversed during query execution – see Section 3.1 for a more detailed exposition). It is unclear how the CS scheme would even function once this leakage is suppressed. A similar argument applies to the FJK+ and the HLK schemes and their leakage profiles.

More generally, suppressing volume leakage in efficient substring-SSE schemes appears to be significantly more challenging than in SSE for keyword search. In SSE for keyword search, one needs to build an (encrypted) index that allows mapping a query keyword to the documents that contain it. However, in substring-SSE, one needs to build an (encrypted)

index that not only allows mapping a query substring to the strings containing it, but also to the precise positions within each string where the queried substring occurs. This creates significantly more avenues for leakage of volume information. In fact, for all the schemes we consider in this paper, the server crucially uses this volume information to optimize the search procedure. A possible approach to design countermeasures to our attacks could be to identify alternative data structures that allow for efficient leakage suppression, and then adapt them to support efficient substring search. We leave a more detailed study of how to to design countermeasures against our proposed attacks as an interesting open question.

Lessons Learned. It is worth noting that we achieve the highest query reconstruction rate for the HLK scheme [36], which was proposed nearly three years after the CS scheme [13] and the FJK+ scheme [20]. This demonstrates that, as far as substring-SSE is concerned, newer schemes are not necessarily "more secure". Another lesson, which is specific to the FJK+ scheme, is that building an SSE scheme for some search functionality (in this case, FJK+ for substring-search) by extending a secure SSE scheme for some other search functionality (in this case, OXT [9] for queries involving boolean formulae over keywords) does not necessarily imply that the resulting scheme inherits the resistance of the original scheme to leakage cryptanalysis. In particular, systematic and functionality-specific leakage cryptanalysis is required regardless of how the scheme is constructed.

More broadly, our attacks demonstrate that, when designing a new SSE scheme, it is not sufficient to merely identify the leakage profile of the designed scheme (which is the case for all existing substring-SSE proposals [4, 13, 20, 36, 52, 56, 57, 69, 71–73]). Rather, it should be incumbent on designers to also cryptanalyze their scheme's leakage profile and show that is not consequential in the face of state-of-the-art attacks. We hope that our novel attack techniques provide a starting point for such analysis for substring-SSE schemes, both old and new.

7 Ethical Considerations

In this paper, we analysed three different SSE schemes offering substring search functionality that were proposed in the literature [13, 20, 36]. We showed that each scheme has serious security shortcomings, in that a fraction of all queries can be recovered by an honest-but-curious server, given access to certain auxiliary information concerning the dataset. It was an explicit aim of each scheme to keep the queries private from the server.

We distinguish broadly between two groups of stakeholders potentially affected by our research: stakeholders that might be impacted specifically by our leakage cryptanalysis of these three schemes, and stakeholders that are impacted more broadly by the technology researched and developed in this paper. We analyse the potential harms and benefits for these two groups using a consequentialist approach.

The most obvious stakeholders potentially impacted by our leakage cryptanalysis study is users of the three schemes we have studied. However, we are not aware of any real-world deployments of any of these three schemes at the time of writing. Indeed, it is only relatively recently that industrial entities have started to add encrypted search capabilities to their

products. As far as we are aware, the only major deployments are in MongoDB's queryable encryption (cf. [35]) and in Microsoft Always Encrypted (cf. [65]).

In September 2025, MongoDB added support for substring queries as a preview feature.[5] Since the feature is rather new, we have not had the opportunity to study it to discover how it works and how it might relate to the three schemes analyzed in this paper. However, the release is heavily caveated in the following way:

> Warning: Queryable Encryption defends against data exfiltration, not against adversaries with persistent access to an environment, or those who can retrieve both database snapshots and accompanying query transcripts/logs.[6]

On the other hand, our attacks do assume an adversary with persistent access to the environment (as we assume the adversary *is* the server and is able to analyse all queries and the corresponding responses, or obtain equivalent information, e.g. from logs). Thus our attacks require stronger capabilities than those possessed by adversaries that MongoDB's product is advertised as defending itself against. It is possible that our attacks, or variants of them, would work against MongoDB's product, but this would not violate the guarantees provided by MongoDB to its customers.

As far as we have been able to determine, Microsoft Always Encrypted does not support substring queries.[7]

We have done basic searches using a search engine and used citation analysis in an effort to discover any real-world deployments of any of the three schemes, but did not identify any. Thus we consider the risk to be low that any users today would be negatively affected by our work. However, we cannot rule out the possibility that some company or organisation somewhere might be using one of the three schemes.

We argue that it is justifiable to use the Enron Email Corpus for leakage cryptanalysis given: (1) this dataset has been used extensively for attack experiments in previous works [3, 38, 59, 60], (2) the corpus has been public for a long time, (3) researchers have attempted to curate the dataset to remove data concerning individuals upon request,[8] (4) individuals could request to have their data redacted if they so wished at any point of time, (5) no specific elements of the dataset are reported in our paper, and (6) our results depend only on statistical properties of the dataset.

Coming now to the broader set of stakeholders, our research contributes to research into searching over outsourced, encrypted databases, by showing that existing schemes in the literature for substring search are insecure in the standard threat model and therefore not suitable for deployment. Our paper should therefore help to ensure that these insecure schemes do not get deployed in future; researchers may also be encouraged by our work to examine the security of further schemes in the literature beyond the three we studied here,

[5] See https://www.mongodb.com/docs/upcoming/core/queryable-encryption/features/#queryable-encryption-features

[6] *Ibid.*

[7] See https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-database-engine?view=sql-server-ver17 for a description of the product features.

[8] See https://www.cs.cmu.edu/~enron/ for detailed documentation.

potentially expanding the set of schemes known to be insecure. Our paper also identifies the need for further research to develop schemes that *are* secure in the standard threat model, and should therefore spur the community to conduct more work in that direction. In these two senses, our paper benefits society at large from the point of view of enhancing privacy of outsourced data.

Given the low risk of adverse effects from our work and the positive benefits that accrue from it (in terms of helping to prevent insecure schemes from being deployed in future), we consider it to be acceptable on balance to have conducted this research and for the work to be made public.

References

[1] Léonard Assouline and Brice Minaud. Weighted oblivious RAM, with applications to searchable symmetric encryption. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part I*, volume 14004 of *Lecture Notes in Computer Science*, pages 426–455, Lyon, France, April 23–27, 2023. Springer, Heidelberg, Germany.

[2] Bieganski, Riedl, Cartis, and Retzel. Generalized suffix trees for biological sequence data: applications and implementation. In *1994 Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences*, volume 5, pages 35–44, 1994.

[3] Laura Blackstone, Seny Kamara, and Tarik Moataz. Revisiting leakage abuse attacks. In *ISOC Network and Distributed System Security Symposium – NDSS 2020*, San Diego, CA, USA, February 23–26, 2020. The Internet Society.

[4] Charlotte Bonte and Ilia Iliashenko. Homomorphic string search with constant multiplicative depth. In Yinqian Zhang and Radu Sion, editors, *ACM SIGSAC CCSW'20*, pages 105–117, 2020.

[5] Raphael Bost. $\Sigma o \phi o \varsigma$: Forward secure searchable encryption. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1143–1154, Vienna, Austria, October 24–28, 2016. ACM Press.

[6] Raphaël Bost, Brice Minaud, and Olga Ohrimenko. Forward and backward private searchable encryption from constrained cryptographic primitives. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 1465–1482, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.

[7] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015: 22nd Conference on Computer and Communications Security*, pages 668–679, Denver, CO, USA, October 12–16, 2015. ACM Press.

[8] David Cash, Joseph Jaeger, Stanislaw Jarecki, Charanjit S. Jutla, Hugo Krawczyk, Marcel-Catalin Rosu, and Michael Steiner. Dynamic searchable encryption in very-large databases: Data structures and implementation. In *ISOC Network and Distributed*

System Security Symposium – NDSS 2014, San Diego, CA, USA, February 23–26, 2014. The Internet Society.

[9] David Cash, Stanislaw Jarecki, Charanjit S. Jutla, Hugo Krawczyk, Marcel-Catalin Rosu, and Michael Steiner. Highly-scalable searchable symmetric encryption with support for Boolean queries. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 353–373, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Heidelberg, Germany.

[10] Javad Ghareh Chamani, Dimitrios Papadopoulos, Charalampos Papamanthou, and Rasool Jalili. New constructions for forward and backward private symmetric searchable encryption. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 1038–1055, Toronto, ON, Canada, October 15–19, 2018. ACM Press.

[11] Yan-Cheng Chang and Michael Mitzenmacher. Privacy preserving keyword searches on remote encrypted data. In John Ioannidis, Angelos Keromytis, and Moti Yung, editors, *ACNS 05: 3rd International Conference on Applied Cryptography and Network Security*, volume 3531 of *Lecture Notes in Computer Science*, pages 442–455, New York, NY, USA, June 7–10, 2005. Springer, Heidelberg, Germany.

[12] Melissa Chase and Seny Kamara. Structured encryption and controlled disclosure. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 577–594, Singapore, December 5–9, 2010. Springer, Heidelberg, Germany.

[13] Melissa Chase and Emily Shen. Substring-searchable symmetric encryption. *Proceedings on Privacy Enhancing Technologies*, 2015(2):263–281, April 2015.

[14] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *36th Annual Symposium on Foundations of Computer Science*, pages 41–50, Milwaukee, Wisconsin, October 23–25, 1995. IEEE Computer Society Press.

[15] Reza Curtmola, Juan A. Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: improved definitions and efficient constructions. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006: 13th Conference on Computer and Communications Security*, pages 79–88, Alexandria, Virginia, USA, October 30 – November 3, 2006. ACM Press.

[16] Marc Damie, Florian Hahn, and Andreas Peter. A highly accurate query-recovery attack against searchable encryption using non-indexed documents. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021: 30th USENIX Security Symposium*, pages 143–160. USENIX Association, August 11–13, 2021.

[17] Ioannis Demertzis, Dimitrios Papadopoulos, Charalampos Papamanthou, and Saurabh Shintre. SEAL: Attack mitigation for encrypted databases via adjustable leakage. In Srdjan Capkun and Franziska Roesner, editors, *USENIX Security 2020: 29th USENIX Security Symposium*, pages 2433–2450. USENIX Association, August 12–14, 2020.

[18] Ioannis Demertzis, Stavros Papadopoulos, Odysseas Papapetrou, Antonios Deligiannakis, and Minos N. Garofalakis. Practical private range search revisited. In *ACM SIGMOD 2016*, pages 185–198, 2016.

[19] Ioannis Demertzis, Stavros Papadopoulos, Odysseas Papapetrou, Antonios Deligiannakis, Minos N. Garofalakis, and Charalampos Papamanthou. Practical private range search in depth. *ACM Trans. Database Syst.*, 43(1):2:1–2:52, 2018.

[20] Sky Faber, Stanislaw Jarecki, Hugo Krawczyk, Quan Nguyen, Marcel-Catalin Rosu, and Michael Steiner. Rich queries on encrypted data: Beyond exact matches. In Günther Pernul, Peter Y. A. Ryan, and Edgar R. Weippl, editors, *ESORICS 2015: 20th European Symposium on Research in Computer Security, Part II*, volume 9327 of *Lecture Notes in Computer Science*, pages 123–145, Vienna, Austria, September 21–25, 2015. Springer, Heidelberg, Germany.

[21] Francesca Falzon, Evangelia Anna Markatou, Akshima, David Cash, Adam Rivkin, Jesse Stern, and Roberto Tamassia. Full database reconstruction in two dimensions. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 443–460, Virtual Event, USA, November 9–13, 2020. ACM Press.

[22] Sanjam Garg, Payman Mohassel, and Charalampos Papamanthou. TWORAM: Efficient oblivious RAM in two rounds with applications to searchable encryption. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 563–592, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Heidelberg, Germany.

[23] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 169–178, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press.

[24] Marilyn George, Seny Kamara, and Tarik Moataz. Structured encryption and dynamic leakage suppression. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part III*, volume 12698 of *Lecture Notes in Computer Science*, pages 370–396, Zagreb, Croatia, October 17–21, 2021. Springer, Heidelberg, Germany.

[25] Eu-Jin Goh. Secure indexes. Cryptology ePrint Archive, Report 2003/216, 2003. https://eprint.iacr.org/2003/216.

[26] Oded Goldreich. Towards a theory of software protection and simulation by oblivious RAMs. In Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 182–194, New York City, NY, USA, May 25–27, 1987. ACM Press.

[27] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious rams. *J. ACM*, 43(3):431–473, 1996.

[28] Paul Grubbs, Anurag Khandelwal, Marie-Sarah Lacharité, Lloyd Brown, Lucy Li, Rachit Agarwal, and Thomas Ristenpart. Pancake: Frequency smoothing for encrypted data stores. In Srdjan Capkun and Franziska Roesner, editors, *USENIX Security 2020: 29th USENIX Security Symposium*, pages 2451–2468. USENIX Association, August 12–14, 2020.

[29] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. Pump up the volume: Practical database reconstruction from volume leakage on range queries. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 315–331, Toronto, ON, Canada, October 15–19, 2018. ACM Press.

[30] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. Learning to reconstruct: Statistical learning theory and encrypted database attacks. In *2019 IEEE Symposium on Security and Privacy*, pages 1067–1083, San Francisco, CA, USA, May 19–23, 2019. IEEE Computer Society Press.

[31] Zichen Gui, Oliver Johnson, and Bogdan Warinschi. Encrypted databases: New volume attacks against range queries. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 361–378, London, UK, November 11–15, 2019. ACM Press.

[32] Zichen Gui, Kenneth G. Paterson, and Sikhar Patranabis. Rethinking searchable symmetric encryption. In *IEEE Symposium on Security and Privacy 2023*, pages 1401–1418. IEEE, 2023.

[33] Zichen Gui, Kenneth G. Paterson, Sikhar Patranabis, and Bogdan Warinschi. SWiSSSE: System-wide security for searchable symmetric encryption. Cryptology ePrint Archive, Report 2020/1328, 2020. https://eprint.iacr.org/2020/1328.

[34] Zichen Gui, Kenneth G. Paterson, Sikhar Patranabis, and Bogdan Warinschi. Swissse: System-wide security for searchable symmetric encryption. *Proc. Priv. Enhancing Technol.*, 2024(1):549–581, 2024.

[35] Zichen Gui, Kenneth G. Paterson, and Tianxin Tang. Security analysis of MongoDB queryable encryption. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 7445–7462, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.

[36] Florian Hahn, Nicolas Loza, and Florian Kerschbaum. Practical and secure substring search. In *SIGMOD 2018*, pages 163–176. ACM, 2018.

[37] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Batch codes and their applications. In László Babai, editor, *36th Annual ACM Symposium on Theory of Computing*, pages 262–271, Chicago, IL, USA, June 13–16, 2004. ACM Press.

[38] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. Access pattern disclosure on searchable encryption: Ramification, attack and mitigation. In *ISOC Network and Distributed System Security Symposium – NDSS 2012*, San Diego, CA, USA, February 5–8, 2012. The Internet Society.

[39] Charanjit S. Jutla and Sikhar Patranabis. Efficient searchable symmetric encryption for join queries. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part III*, volume 13793 of *Lecture Notes in Computer Science*, pages 304–333, Taipei, Taiwan, December 5–9, 2022. Springer, Heidelberg, Germany.

[40] Seny Kamara and Tarik Moataz. Boolean searchable symmetric encryption with worst-case sub-linear complexity. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part III*, volume 10212 of *Lecture Notes in Computer Science*, pages 94–124, Paris, France, April 30 – May 4, 2017. Springer, Heidelberg, Germany.

[41] Seny Kamara and Tarik Moataz. SQL on structurally-encrypted databases. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part I*, volume 11272 of *Lecture Notes in Computer Science*, pages 149–180, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Heidelberg, Germany.

[42] Seny Kamara and Tarik Moataz. Computationally volume-hiding structured encryption. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 183–213, Darmstadt, Germany, May 19–23, 2019. Springer, Heidelberg, Germany.

[43] Seny Kamara, Charalampos Papamanthou, and Tom Roeder. Dynamic searchable symmetric encryption. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012: 19th Conference on Computer and Communications Security*, pages 965–976, Raleigh, NC, USA, October 16–18, 2012. ACM Press.

[44] Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O'Neill. Generic attacks on secure outsourced databases. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1329–1340, Vienna, Austria, October 24–28, 2016. ACM Press.

[45] Evgenios M. Kornaropoulos, Nathaniel Moyer, Charalampos Papamanthou, and Alexandros Psomas. Leakage inversion: Towards quantifying privacy in searchable encryption. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 1829–1842, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.

[46] Evgenios M. Kornaropoulos, Charalampos Papamanthou, and Roberto Tamassia. Data recovery on encrypted databases with k-nearest neighbor query leakage. In *2019 IEEE Symposium on Security and Privacy*, pages 1033–1050, San Francisco, CA, USA, May 19–23, 2019. IEEE Computer Society Press.

[47] Evgenios M. Kornaropoulos, Charalampos Papamanthou, and Roberto Tamassia. The state of the uniform: Attacks on encrypted databases beyond the uniform query distribution. In *2020 IEEE Symposium on Security and Privacy*, pages 1223–1240, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press.

[48] Evgenios M. Kornaropoulos, Charalampos Papamanthou, and Roberto Tamassia. Response-hiding encrypted ranges: Revisiting security via parametrized leakage-abuse attacks. In *2021 IEEE Symposium on Security and Privacy*, pages 1502–1519, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.

[49] Eyal Kushilevitz and Rafail Ostrovsky. Replication is NOT needed: SINGLE database, computationally-private information retrieval. In *38th Annual Symposium on Foundations of Computer Science*, pages 364–373, Miami Beach, Florida, October 19–22, 1997. IEEE Computer Society Press.

[50] Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. Improved reconstruction attacks on encrypted data using range query leakage. In *2018 IEEE Symposium on Security and Privacy*, pages 297–314, San Francisco, CA, USA, May 21–23, 2018. IEEE Computer Society Press.

[51] Shangqi Lai, Sikhar Patranabis, Amin Sakzad, Joseph K. Liu, Debdeep Mukhopadhyay, Ron Steinfeld, Shifeng Sun, Dongxi Liu, and Cong Zuo. Result pattern hiding searchable encryption for conjunctive queries. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 745–762, Toronto, ON, Canada, October 15–19, 2018. ACM Press.

[52] Iraklis Leontiadis and Ming Li. Storage efficient substring searchable symmetric encryption. In Aziz Mohaisen and Qian Wang, editors, *Proceedings of the 6th International Workshop on Security in Cloud Computing, SCC@AsiaCCS*, pages 3–13. ACM, 2018.

[53] Feng Li, Jianfeng Ma, Yinbin Miao, Ximeng Liu, Jianting Ning, and Robert H. Deng. A survey on searchable symmetric encryption. *ACM Comput. Surv.*, 56(5):119:1–119:42, 2024.

[54] Evangelia Anna Markatou, Francesca Falzon, Roberto Tamassia, and William Schor. Reconstructing with less: Leakage abuse attacks in two dimensions. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2243–2261, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.

[55] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *2022 IEEE Symposium on Security and Privacy*, pages 930–947, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.

[56] Tarik Moataz and Erik-Oliver Blass. Oblivious substring search with updates. Cryptology ePrint Archive, Report 2015/722, 2015. https://eprint.iacr.org/2015/722.

[57] Tarik Moataz, Indrajit Ray, Indrakshi Ray, Abdullatif Shikfa, Frédéric Cuppens, and Nora Cuppens. Substring search over encrypted data. *J. Comput. Secur.*, 26(1):1–30, 2018.

[58] Jianting Ning, Xinyi Huang, Geong Sen Poh, Jiaming Yuan, Yingjiu Li, Jian Weng, and Robert H. Deng. LEAP: Leakage-abuse attack on efficiently deployable, efficiently searchable encryption with partially known dataset. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2307–2320, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.

[59] Simon Oya and Florian Kerschbaum. Hiding the access pattern is not enough: Exploiting search pattern leakage in searchable encryption. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021: 30th USENIX Security Symposium*, pages 127–142. USENIX Association, August 11–13, 2021.

[60] Simon Oya and Florian Kerschbaum. IHOP: Improved statistical query recovery against searchable symmetric encryption through quadratic optimization. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022: 31st USENIX Security Symposium*, pages 2407–2424, Boston, MA, USA, August 10–12, 2022. USENIX Association.

[61] Sarvar Patel, Giuseppe Persiano, Joon Young Seo, and Kevin Yeo. Efficient boolean search over encrypted data with reduced leakage. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 577–607, Singapore, December 6–10, 2021. Springer, Heidelberg, Germany.

[62] Sarvar Patel, Giuseppe Persiano, Kevin Yeo, and Moti Yung. Mitigating leakage in secure cloud-hosted data structures: Volume-hiding for multi-maps via hashing. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 79–93, London, UK, November 11–15, 2019. ACM Press.

[63] Sikhar Patranabis and Debdeep Mukhopadhyay. Forward and backward private conjunctive searchable symmetric encryption. In *ISOC Network and Distributed System Security Symposium – NDSS 2021*, Virtual, February 21–25, 2021. The Internet Society.

[64] David Pouliot and Charles V. Wright. The shadow nemesis: Inference attacks on efficiently deployable, efficiently searchable encryption. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1341–1352, Vienna, Austria, October 24–28, 2016. ACM Press.

[65] Ryan Seah, Daren Khu, Alexander Hoover, and Ruth Ng. LAMA: leakage abuse attacks against microsoft always encrypted. In Sabrina De Capitani di Vimercati and Pierangela Samarati, editors, *Proceedings of the 21st International Conference on Security and Cryptography, SECRYPT 2024, Dijon, France, July 8-10, 2024*, pages 628–633. SCITEPRESS, 2024.

[66] Elaine Shi, T.-H. Hubert Chan, Emil Stefanov, and Mingfei Li. Oblivious RAM with $O((\log N)^3)$ worst-case cost. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 197–214, Seoul, South Korea, December 4–8, 2011. Springer, Heidelberg, Germany.

[67] Dawn Xiaodong Song, David Wagner, and Adrian Perrig. Practical techniques for searches on encrypted data. In *2000 IEEE Symposium on Security and Privacy*, pages 44–55, Oakland, CA, USA, May 2000. IEEE Computer Society Press.

[68] Emil Stefanov, Marten van Dijk, Elaine Shi, Christopher W. Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM: an extremely simple oblivious RAM protocol. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013: 20th Conference on Computer and Communications Security*, pages 299–310, Berlin, Germany, November 4–8, 2013. ACM Press.

[69] Mikhail Strizhov and Indrajit Ray. Substring position search over encrypted cloud data using tree-based index. In *2015 IEEE International Conference on Cloud Engineering, IC2E*, pages 165–174. IEEE Computer Society, 2015.

[70] Peter J. van Laarhoven. *Simulated annealing theory and applications*. Kluwer, 1987.

[71] Hiroaki Yamamoto. Secure automata-based substring search scheme on encrypted data. In Kazuto Ogawa and Katsunari Yoshioka, editors, *IWSEC 16: 11th International Workshop on Security, Advances in Information and Computer Security*, volume 9836 of *Lecture Notes in Computer Science*, pages 111–131, Tokyo, Japan, September 12–14, 2016. Springer, Heidelberg, Germany.

[72] Hiroaki Yamamoto, Yoshihiro Wachi, and Hiroshi Fujiwara. Space-efficient and secure substring searchable symmetric encryption using an improved DAWG. In *Provable Security - 13th International Conference, ProvSec 2019*, volume 11821, pages 130–148. Springer, 2019.

[73] Fan Yin, Rongxing Lu, Yandong Zheng, Jun Shao, Xue Yang, and Xiaohu Tang. Achieve efficient position-heap-based privacy-preserving substring-of-keyword query over cloud. *Comput. Secur.*, 110:102432, 2021.

A Additional Preliminaries

Correctness of Substring-SSE. We say that a substring-SSE scheme Π is correct if **EQuery** returns the correct substring matching results. That is, for any database **DB** and any query q, when executing the following sequence:

1. $\mathsf{sk} \leftarrow \mathbf{Gen}_{\mathbf{Clt}}(1^\lambda)$
2. $(\perp, \mathbf{EDB}) \leftarrow [\mathbf{Setup}_{\mathbf{Clt}}(\mathsf{sk}, \mathbf{DB}), \mathbf{Setup}_{\mathbf{Svr}}()]$
3. $((\mathsf{ind}_i, \mathsf{pos}_i)_{i=1}^l, \perp) \leftarrow [\mathbf{EQuery}_{\mathbf{Clt}}(\mathsf{sk}, q), \mathbf{EQuery}_{\mathbf{Svr}}(\mathbf{EDB})]$

we have $(\mathsf{ind}_i, \mathsf{pos}_i)_{i=1}^l = \mathbf{Query}(\mathbf{DB}, q)$.

Security of Substring-SSE. We define simulation-based security of substring-SSE against a semi-honest adversarial server (i.e., the server follows the specification of the scheme, but tries to learn information about the underlying plaintext database and queries). Our definition is in the real-world, ideal-world paradigm, and closely follows the traditional simulation-based security definitions that are widely used in the SSE literature [8, 9, 12, 15]. Let $\Pi = (\mathbf{Gen}, \mathbf{Setup}, \mathbf{EQuery})$ be a substring-SSE scheme. Consider the following probabilistic experiments where $\mathcal{A}$ is a stateful probabilistic polynomial-time (PPT) adversary, $\mathcal{S}$ is a PPT simulator, and $\mathcal{L} = (\mathcal{L}_{\mathbf{Setup}}, \mathcal{L}_{\mathbf{EQuery}})$ is a stateful leakage function:

- $\mathbf{Real}_{\Pi,\mathcal{A}}(1^\lambda)$: the challenger begins by running $\mathbf{Gen}_{\mathbf{Clt}}(1^\lambda)$ to generate a secret key sk. The adversary $\mathcal{A}$ outputs a database **DB**. The challenger and the adversary $\mathcal{A}$ interact to output $(\perp, \mathbf{EDB}) \leftarrow [\mathbf{Setup}_{\mathbf{Clt}}(\mathsf{sk}, \mathbf{DB}), \mathbf{Setup}_{\mathbf{Svr}}()]$, where the challenger plays the client and $\mathcal{A}$ plays the server. The adversary $\mathcal{A}$ then makes a polynomial number of adaptive substring matching queries: each query involves running $[\mathbf{EQuery}_{\mathbf{Clt}}(\mathsf{sk}, q), \mathbf{EQuery}_{\mathbf{Svr}}(\mathbf{EDB})]$ on an adversarially chosen q, with the challenger acting as the client **Clt** and $\mathcal{A}$ acting as the server. Finally, $\mathcal{A}$ returns a bit b that is output by the experiment.

- $\mathbf{Ideal}_{\Pi,\mathcal{A},\mathcal{S}}(1^\lambda)$: The adversary $\mathcal{A}$ outputs a database $\mathbf{DB}$. The simulator $\mathcal{S}$ and the adversary $\mathcal{A}$ interact to output $(\perp, \mathbf{EDB}) \leftarrow [\mathcal{S}(\mathcal{L}_{\mathbf{Setup}}(\mathbf{DB})), \mathbf{Setup}_{\mathbf{Svr}}()]$, where $\mathcal{S}$ plays the client and $\mathcal{A}$ plays the server. The adversary $\mathcal{A}$ then makes a polynomial number of adaptive substring matching queries: each query involves running the *simulated* query protocol $[\mathcal{S}(\mathcal{L}_{\mathbf{EQuery}}(q)), \mathbf{EQuery}_{\mathbf{Svr}}(\mathbf{EDB})]$ on an adversarially chosen q, with the simulator $\mathcal{S}$ acting as the client $\mathbf{Clt}$ and $\mathcal{A}$ acting as the server. Finally, $\mathcal{A}$ returns a bit b that is output by the experiment.

We say that a substring-SSE scheme Π is $\mathcal{L}$-secure against adaptive chosen-query attacks if for every PPT adversary $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that

$$\left|\mathbf{Pr}[\mathbf{Real}_{\Pi,\mathcal{A}}(1^\lambda) = 1] - \mathbf{Pr}[\mathbf{Ideal}_{\Pi,\mathcal{A},\mathcal{S}}(1^\lambda) = 1]\right| \leq \mathsf{negl}(\lambda).$$

Inference-style Leakage Cryptanalysis. Note that the above security definition informally captures the fact that a semi-honest adversarial server does not learn any information about the client's plaintext database and queries beyond what is captured by the leakage function $\mathcal{L} = (\mathcal{L}_{\mathbf{Setup}}, \mathcal{L}_{\mathbf{EQuery}})$. In this paper, we design query reconstruction attacks where a semi-honest adversary exploits the leakage function $\mathcal{L}$ to recover the client's queries. All of our attacks are *inference attacks*. More specifically, all of our attacks assume that the attacker has access to some auxiliary statistical information about the underlying plaintext database. Given this statistical information about the database and the transcript of encrypted query executions, the semi-honest attacker attempts to infer the underlying plaintext queries.

Semi-Honest vs Malicious Corruptions. We remark here that Chase and Shen [13] originally claimed security of their CS scheme in a *strictly stronger* model of security for substring-SSE where the server is allowed to be *maliciously* corrupt. In the security definitions above, we only consider *semi-honest* corruptions of the server, which is a weaker adversarial model (besides this, the security definition above is the same as that in [13]). We choose to present the security definition for substring-SSE in the semi-honest adversarial model, since this is the most widely studied setting in the SSE literature [8,9,12,15]. This is also consistent with our proposed attacks, each of which assumes a semi-honest adversary.

B Full Description of the CS Scheme

In this section, we show how Chase and Shen [13] built their substring-SSE scheme CS from suffix trees. Following their presentation, we show three attempts at building a substring-SSE scheme.

A First Attempt. The first idea from [13] is as follows. Let F be a pseudorandom function and Γ be a CPA-secure symmetric encryption scheme. Let sk_1 be a key for F and sk_2 be a key for Γ. Let node N be a non-root node of the suffix tree. The substring-SSE scheme encrypts compute a PRF value $t = \mathsf{F}_{\mathsf{sk}_1}(\mathsf{path}(\mathsf{N}))$ and encrypts the index stored in node N as $c = \Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(\mathsf{ind}(\mathsf{N}))$. The scheme stores (t, c) in an encrypted dictionary D where t is used as the key and c is used as the value. This process is performed on all non-root nodes.

This scheme allows the client to query all substrings that are paths in the suffix tree by using encrypted queries of the form $eq = \mathsf{F}_{\mathsf{sk}_1}(q)$. If q is one of the paths, then the PRF value $\mathsf{F}_{\mathsf{sk}_1}(q)$ is in the encrypted dictionary D, and the value stored in $\mathsf{D}[t]$ is an encryption of the index of the first match for q. Note that this scheme only returns the index of the first match since tree traversal is impossible with the encrypted dictionary D.

Returning a Possible Match. Define $\mathsf{initpath}(\mathsf{N})$ as the concatenation of strings on the edges from the root to node N as before, except that for the last edge (closest to N), only the first character of the string on the edge is used. For example, in Figure 1, $\mathsf{initpath}(\mathsf{N}_7) =$ "hell" and $\mathsf{initpath}(\mathsf{N}_9) =$ "ell".

In the second attempt, Chase and Shen make the following changes. Instead of querying node N with $\mathsf{F}_{\mathsf{sk}_1}(\mathsf{path}(\mathsf{N}))$, we query node $\mathsf{initpath}(\mathsf{N})$. This can be done by computing the encrypted query as $eq = (\mathsf{F}_{\mathsf{sk}_1}(q[1,1]), \ldots, \mathsf{F}_{\mathsf{sk}_1}(q[1,|q|]))$. Given eq, the server finds the PRF value with the longest substring of q in the encrypted dictionary D and returns the value associated with it. The client then decrypts the value to obtain a string index (i,j). After that, the client sends $(i, j|q|)$ to the server, retrieves the following encrypted values (these should be stored on the server in the **Setup** phase):

$$\Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(S_i[j]), \ldots, \Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(S_i[j+|q|-1]),$$

and checks if the decryptions of the characters are the same as q. If the characters match, then (i,j) is an index for the match. Otherwise, q does not have any match in the database.

Returning All Occurrences. A trivial solution to allow the client to retrieve all matching occurrences of a query is to store all matching indices in the encrypted dictionary. However, this solution has a linear storage blow-up (with respect to the lengths of the strings the server stores) in the worst case.

To overcome this problem, Chase and Shen used the fact that if node N is the node that matches a query in the second attempt, all matching indices of the query must be exactly the indices stored in the leaf nodes in the subtree of N. Hence, it suffices to store the indexing information of the subtree of N in N, so that the leaf nodes can be accessed later.

This is done as follows. Let leaf_i be the i-th leaf node of the suffix tree. In the **Setup** phase, the client creates an encrypted array L where $\mathsf{L}[i] = \Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(\mathsf{ind}(\mathsf{leaf}_i))$. This is the same index as the encrypted dictionary D stores for the leaf nodes in the second attempt. As for the encrypted dictionary D, in addition to the index of the first occurrence, we also store the subtree information for every entry. For node N, define $\mathsf{leafpos}(\mathsf{N})$ as the position of the leftmost leaf node in the subtree of N. Define $\mathsf{num}(\mathsf{N})$ as the number of leaf nodes in the subtree of N.[9] Then, for the entry with dictionary key $\mathsf{F}_{\mathsf{sk}_1}(\mathsf{initpath}(\mathsf{N}))$, we store $\Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(\mathsf{ind}(\mathsf{N}), \mathsf{leafpos}(\mathsf{N}), \mathsf{num}(\mathsf{N}))$.

An encrypted substring query will now proceed as follows. Let q be a query string. The client begins by computing $eq = (\mathsf{F}_{\mathsf{sk}_1}(q[1,1]), \ldots, \mathsf{F}_{\mathsf{sk}_1}(q[1,|q|]))$ and sending it to the server. The server finds the PRF value with the longest substring of q in the encrypted dictionary D and returns the value associated with it. The client decrypts the value and get

[9] See Figure 1 for an example.

$\mathsf{ind}(\mathsf{N}), \mathsf{leafpos}(\mathsf{N}), \mathsf{num}(\mathsf{N})$ for some node N that is a prefix of q. The client retrieves

$$\Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(S_i[j]), \ldots, \Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(S_i[j+|q|-1]),$$

and checks if the decryptions of the characters are the same as q. If they are the same, the client retrieves

$$\mathsf{L}[\mathsf{leafpos}(\mathsf{N})], \ldots, \mathsf{L}[\mathsf{leafpos}(\mathsf{N}) + \mathsf{num}(\mathsf{N}) - 1],$$

and decrypts them to obtain all indices of matching occurrences. Otherwise, the client knows that there are no matching occurrences for query q.

Further Modifications. The final CS scheme makes the following modifications to the third attempt to reduce its leakage:

- The search tokens are encrypted and the content of the encrypted dictionary is modified so that the scheme only leaks information when two queries have a shared prefix.
- Node degrees and the number of nodes in the suffix tree are obfuscated by padding. The order of children nodes are hidden by permuting them.
- The string indices are hidden by permuting the encrypted ciphertexts, i.e.,

$$\Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(S_1[1]), \ldots, \Gamma.\mathsf{Enc}_{\mathsf{sk}_2}(S_N[|S_N|]).$$

C Leakage Profiles of the Schemes

C.1 Leakage Profile of the CS Scheme

In this section, we describe the leakage profile of the CS scheme. We include an example to illustrate the leakage.

Leakage Description. During **Setup**, the scheme leaks the total length of the strings. That is, if $\mathbf{DB} = (S_1, \ldots, S_N)$, then we have

$$\mathcal{L}_{\mathbf{Setup}}(\mathbf{DB}) = \sum_{i=1}^{N} |S_i|.$$

We now focus on the leakage during queries, i.e., $\mathcal{L}_{\mathbf{EQuery}}$. For a substring query q, let $\mathsf{initpath}(\mathsf{N})$ where N is a node in the suffix tree T be the longest prefix of q. When processing a query q, the scheme leaks the length of the query $|q|$ and the length of the prefix $|\mathsf{initpath}(\mathsf{N})|$. For a sequence of queries $q_1, \ldots, q_l$, the CS scheme leaks three additional patterns:

- *The query prefix pattern* $\mathsf{QP}(\mathbf{DB}, q_1, \ldots, q_l)$ for query q_l indicates for every node visited by query q_l, if the node is visited by any of the previous queries. The pattern can be represented by an $l \times n_i$ matrix, where n_i is the number of nodes visited by query q_l. The (i, j)-th entry of the matrix is 1 if q_i visited the j-th node that is visited by query q_l; the entry is 0 otherwise.

- *The leaf intersection pattern* $\mathsf{LP}(\mathbf{DB}, q_1, \ldots, q_l)$ for query q_l indicates which leaf indices retrieved by query q_l are also retrieved by previous queries. The pattern can be represented by an $l \times m_j$ matrix, where m_j is the number of leaf nodes retrieved by query q_l. Let $r_1 : [m_j] \to [m_j]$ be a random permutation. The (i, j)-th entry of the leaf intersection pattern matrix is 1 if the $r_i(i)$-th leaf retrieved by query q_l is also retrieved by query q_i; the entry is 0 otherwise.
- *The index intersection pattern* $\mathsf{IP}(\mathbf{DB}, q_1, \ldots, q_l)$ for query q_l indicates which string indices retrieved by query q_l are also retrieved by previous queries. The pattern can be represented by an $l \times |q_l|$ matrix. Let $r_2 : [|q_l|] \to [|q_l|]$ be a random permutation. The (i, j)-th entry of the index intersection pattern matrix is 1 if the i-th string index retrieved by query q_l is also retrieved by query q_i; the entry is 0 otherwise.

Leakage Illustration using an Example. To illustrate the leakage of the scheme, consider the database $\mathbf{DB}$ = ("hello", "help"). The server learns that there are 9 leaf nodes from $|\mathsf{L}|$, which is exactly the total length of the strings.

Now, consider query q_1 = "ello" and q_2 = "elp". The server learns that $|q_1| = 4$ and $|q_2| = 3$ since the server sees 4 and 3 encrypted PRF values, respectively. The server also learns that $|(\mathsf{initpath}(\text{"ello"}))| = |\text{"ell"}| = 3$ and $|(\mathsf{initpath}(\text{"elp"}))| = |\text{"elp"}| = 3$ since the PRF values used to retrieve the nodes associated to these initial paths reveal that the inputs to the PRF have length 3. When looking at q_1 and q_2 together, we see that a semi-honest adversary can immediately infer the following:

- *The query prefix pattern:* The server sees node N_3 is a shared prefix for both queries since it is visited by both queries. The server also sees that $|\mathsf{initpath}(\mathsf{N}_3))| = 1$ since it has to be retrieved with $\mathsf{F}_{\mathsf{sk}_1}(q_1[1, 1])$ or $\mathsf{F}_{\mathsf{sk}_1}(q_2[1, 1])$.
- *The leaf intersection pattern:* The server does not see any leaf intersection since the two queries visit disjoint sets of nodes ($\{\mathsf{N}_9\}$ and $\{\mathsf{N}_{10}\}$ respectively). However, the server can infer that both queries have query response volume 1.
- *The index intersection pattern:* The server does not see any index intersection since the first string is on the first occurrence of $\mathsf{initpath}(q_1)$ and the second string is on the first occurrence of $\mathsf{initpath}(q_2)$.

C.2 Leakage Profile of the FJK+ Scheme

In this section, we give a description of the leakage of the FJK+ scheme as presented in the original paper [20]. We assume each query contains two k-grams only (one s-term and one x-term) for simplicity. The leakage profile of the scheme is a tuple $(N, \bar{s}, \mathsf{SP}, \mathsf{RP}, \mathsf{DP}, \mathsf{IP})$, where

- N is the total number of k-grams in all documents.
- $\bar{s}$ is the equality pattern between the s-terms of the substring queries.
- SP is the s-term support size, i.e., the number of times each s-term appears in the database. This is leaked from the first data structure described above.

- RP is the results pattern. It contains all $(\mathsf{ind}, \mathsf{pos})$ pairs which match the whole query.
- DP is the delta pattern of the queries. It contains the offsets of the x-terms in the queries.
- IP is the conditional intersection pattern. It contains the following information. Given two queries $(\mathsf{kg}_1, (\Delta_2, \mathsf{kg}_2))$ and $(\mathsf{kg}_3, (\Delta_4, \mathsf{kg}_4))$, if $\mathsf{kg}_2 = \mathsf{kg}_4$ (i.e., the x-terms are the same), and if there exists at least one location $(\mathsf{ind}, \mathsf{pos})$ such that $(\mathsf{kg}_2, \mathsf{ind}, \mathsf{pos}) \in \mathbf{DB}$, $(\mathsf{kg}_1, \mathsf{ind}, \mathsf{pos} - \Delta_2) \in \mathbf{DB}$, and $(\mathsf{kg}_3, \mathsf{ind}, \mathsf{pos} - \Delta_4) \in \mathbf{DB}$, then the conditional intersection pattern between the two queries are $(\mathsf{ind}, \mathsf{pos} - \Delta_2, \mathsf{pos} - \Delta_4)$. Otherwise, the conditional intersection pattern is empty. We note that the leakage is over-specified as the scheme does not leak $(\mathsf{ind}, \mathsf{pos})$ directly. However, the server can learn the overlap between the s-terms (kg_1 and kg_3) for queries where their conditional intersection pattern is non-empty. This will be demonstrated with an example below.

The reason for the over-specified leakage profile (i.e, it contains some information which the server does not learn, such as the plaintext indices and positions in RP) is for ease in constructing the security proof. In our attack, we only use the leakage components which a real server can learn. See Section 4.2 for more details.

A Concrete Example. We demonstrate the leakage of the scheme for generic queries (i.e., with multiple x-terms) with a concrete example. From now on, we assume all of the k-grams are 3-grams. Consider the following queries:

- $q_1 = (\mathsf{kg}_1, (\Delta_2, \mathsf{kg}_2)) = (\text{'mot'}, (3, \text{'her'}))$,
- $q_2 = (\mathsf{kg}_3, (\Delta_4, \mathsf{kg}_4)) = (\text{'mot'}, (3, \text{'ion'}))$,
- $q_3 = (\mathsf{kg}_5, (\Delta_6, \mathsf{kg}_6), (\Delta_7, \mathsf{kg}_7)) = (\text{'oth'}, (2, \text{'her'}), (5, \text{'wis'}))$.

The server learns N from the setup of the encrypted database. For the other leakage components, the server learns:

- $\bar{s}$: The s-term of q_1 and q_2 are the same.
- SP: The server learns the number of times 'mot' and 'oth' appears in the documents respectively.
- RP: The server learns the number of documents matching 'mother', 'motion', 'other', and 'oth**wis'. The '*' in 'oth**wis' indicates a wildcard. Crucially, the results pattern for general queries reveal the number of matching locations in the documents for each s-term-x-term pair in the queries.
- DP: The server learns all the offsets of the queries.
- IP: If there is a document containing 'mother', the server will learn $\mathsf{kg}_2 = \mathsf{kg}_6$, and the second and third characters of kg_1 are the same as the first and second characters of kg_5.

Algorithm 1 Identifying Candidate Sets

Parameters: Parameter for the initial identification of the candidate sets σ, parameter for trimming the candidate sets t.

Input: Query response volumes $(\mathsf{vol}_i)_{i=1}^{l}$, sequences of tokens extracted from the encrypted queries $(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}$, auxiliary distribution Aux.

Output: Candidate set $\mathsf{CandSet} : \mathbb{N}^* \to \mathcal{P}(\Sigma^*)$ that maps each sequence of tokens to a set of plaintext strings.

Candidate_Set$((\mathsf{vol}_i)_{i=1}^{l}, (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}, \mathsf{Aux})$

Initialise $\mathsf{CandSet}$ as an empty map
for $i \leftarrow 1, \ldots, l$ **do**
    $\mathsf{CandSet}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})) \leftarrow \{s \in \mathsf{Aux} \mid |s| = m_1 \wedge |\mathsf{vol}_i - \mathsf{Aux}(s)| \leq \sigma \cdot \sqrt{\mathsf{vol}_i}\}$ ▷ Initial filtering of candidate sets based on query response volumes. $s \in \mathsf{Aux}$ means getting the strings used to index Aux.
$\mathsf{CandSet}' \leftarrow \mathsf{CandSet}$
$\mathsf{CandSet} \leftarrow$ **Trim_Candidates**$(\mathsf{CandSet})$
while $\mathsf{CandSet}' \neq \mathsf{CandSet}$ **do**
    $\mathsf{CandSet}' \leftarrow \mathsf{CandSet}$
    $\mathsf{CandSet} \leftarrow$ **Trim_Candidates**$(\mathsf{CandSet})$
return $\mathsf{CandSet}$

Trim_Candidates$(\mathsf{CandSet})$
Initialise $Arr : \mathbb{N} \times \Sigma \to \mathbb{N}$ as an array filled with zeros
for $i \in 1, \ldots, l$ **do**
    for $j \in 1, \ldots, m_i$ **do**
        for $c \in \Sigma$ **do**
            if $\exists$ $\mathsf{cand} \in \mathsf{CandSet}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))$ such that $\mathsf{cand}[j] = c$ **then**
                $Arr(\mathsf{tk}_{i,j}, \mathsf{cand}[j]) \leftarrow Arr(\mathsf{tk}_{i,j}, \mathsf{cand}[j]) + 1$
Initialise $\mathsf{TokenCandSet} : \mathbb{N} \to \mathcal{P}(\Sigma)$ as an empty map
for $\mathsf{tk} \in \mathsf{tk}_{1,1}, \ldots, \mathsf{tk}_{l,m_l}$ **do**
    Let $c_1, \ldots, c_k$ be the second input of Arr sorted by decreasing $Arr(\mathsf{tk}, \cdot)$
    $\mathsf{TokenCandSet}(\mathsf{tk}) \leftarrow \{c_1, \ldots, c_{\mathsf{t}}\}$
    if $Arr(\mathsf{tk}, c_{\mathsf{t}}) = Arr(\mathsf{tk}, c_{\mathsf{t}+1})$ **then** ▷ Handling the special case where more than t characters have the same frequency in $Arr(\mathsf{tk}, \cdot)$
        $j \leftarrow \mathsf{t}$
        while $Arr(\mathsf{tk}, c_j) = Arr(\mathsf{tk}, c_{j+1})$ **do**
            $\mathsf{TokenCandSet}(\mathsf{tk}) \leftarrow \mathsf{TokenCandSet}(\mathsf{tk}) \cup \{c_{j+1}\}$
$\mathsf{CandSet}' \leftarrow \{\}$
for $(\mathsf{tk}_1, \ldots, \mathsf{tk}_l) \in \mathsf{CandSet}$ **do**
    $\mathsf{CandSet}(\mathsf{cand}) = \{(c_1, \ldots, c_l) \in \mathsf{CandSet}((\mathsf{tk}_1, \ldots, \mathsf{tk}_l)) \mid c_i \in \mathsf{TokenCandSet}(\mathsf{tk}_i) \forall i\}$
return $\mathsf{CandSet}'$

D Full Attack Pseudocodes

D.1 Attack Pseudocode For Our Attack Against the CS Scheme

Algorithm 1 provides pseudocode for the identification of candidates in our attack against the CS scheme. Algorithm 2 describes our simulated annealing steps in the attack.

Algorithm 2 Simulated Annealing-based Attack Procedure

Parameters: Initial temperature T_{init}, the number of iterations $\mathsf{iter}_{\mathsf{max}}$.

Input: Query response volumes $(\mathsf{vol}_i)_{i=1}^{l}$, sequences of tokens extracted from the encrypted queries $(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}$, candidate set $\mathsf{CandSet}$, auxiliary distribution Aux, the maximum number of iterations for simulated annealing $\mathsf{iter}_{\mathsf{max}}$.

Output: A guess for the tokens of the encrypted queries $\mathsf{guess} : \mathbb{N} \to \Sigma$.

Simulated_annealing$((\mathsf{vol}_i)_{i=1}^{l}, (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}, \mathsf{CandSet}, \mathsf{Aux})$:

$\mathsf{guess} \leftarrow$ **Initial_solution**$(\mathsf{CandSet})$
$\mathsf{T} \leftarrow \mathsf{T}_{init}$
for $i \leftarrow 1, \ldots, \mathsf{iter}_{\mathsf{max}}$ **do**
  $\mathsf{T} \leftarrow$ **Cooling**(T)
  $\mathsf{guess}' \leftarrow$ **Neighbour**$(\mathsf{guess}, \mathsf{CandSet})$
  $\mathsf{sc} \leftarrow$ **Score**$(\mathsf{guess}, (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}, (\mathsf{vol}_i)_{i=1}^{l}, \mathsf{Aux})$
  $\mathsf{sc}' \leftarrow$ **Score**$(\mathsf{guess}', (\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}, (\mathsf{vol}_i)_{i=1}^{l}, \mathsf{Aux})$
  if **Accept_prob**$(\mathsf{sc}, \mathsf{sc}', \mathsf{T}) == 1$ **then**
    $\mathsf{guess} \leftarrow \mathsf{guess}'$
return guess

Initial_solution$(\mathsf{CandSet})$:

$\mathsf{guess} \leftarrow \{\}$
for $(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}) \in \mathsf{CandSet}$ **do** ▷ $t \in \mathsf{CandSet}$ means getting the keys of the map.
  $\mathsf{cand} \xleftarrow{\$} \mathsf{CandSet}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))$
  for $j \in 1, \ldots, m_i$ **do**
    $\mathsf{guess}(\mathsf{tk}_{i,j}) \leftarrow \mathsf{cand}[j]$
  return guess

Neighbour$(\mathsf{guess}, \mathsf{CandSet})$:

$\mathsf{guess}' \leftarrow \mathsf{guess}$
$(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}) \xleftarrow{\$} \mathsf{CandSet}$ ▷ $t \xleftarrow{\$} \mathsf{CandSet}$ means sampling a random key from the map.
$\mathsf{cand} \xleftarrow{\$} \mathsf{CandSet}((\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i}))$
for $j \in 1, \ldots, m_i$ **do**
  $\mathsf{guess}'(\mathsf{tk}_{i,j}) \leftarrow \mathsf{cand}[j]$
return guess'

Accept_prob$(\mathsf{sc}, \mathsf{sc}', \mathsf{T})$:

return $\exp\left\{\frac{\mathsf{sc}'-\mathsf{sc}}{\mathsf{T}}\right\} >$ **rand**$(0,1)$ ▷ **rand**$(0,1)$ is an uniform random variable with range from 0 to 1.

D.2 Attack Pseudocode For Our Attack Against the FJK+ Scheme

Algorithm 3 provides pseudocode for the identification of s-term candidates in our attack against the FJK+ scheme, while Algorithms 4 and 5 describe our first and second rounds of simulated annealing in the attack.

Algorithm 3 Identifying the s-term candidates

Parameters: Parameter for trimming the candidate sets for the s-terms t_s.

Input: Leakage representations $\{\mathsf{lk}_1, \ldots, \mathsf{lk}_n\}$, volume maps $(\mathsf{vol}_s, \mathsf{vol}_x)$, conditional intersection pattern SIP and auxiliary distributions $(\mathsf{Aux}_s, \mathsf{Aux}_x)$.

Output: Candidates for each x-term $\mathsf{CandSet}_s : \mathbb{N} \to \mathcal{P}(\Sigma^3)$ (assuming 3-grams for simplicity).

Candidate_Set$(\{\mathsf{lk}_1, \ldots, \mathsf{lk}_n\}, (\mathsf{vol}_s, \mathsf{vol}_x), \mathsf{SIP}, (\mathsf{Aux}_s, \mathsf{Aux}_x))$

Build the candidate sets for the s-terms using volume leakage only

$\mathsf{CandSet}_s \leftarrow \{\}$
for $(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l)) \in \{\mathsf{lk}_1, \ldots, \mathsf{lk}_n\}$ **do**
    $\mathsf{CandSet}_s(\mathsf{sid}) \leftarrow \{\mathsf{kg} \mid \mathsf{t}_s[1] \cdot \mathsf{vol}_s(\mathsf{sid}) \leq \mathsf{Aux}_s(\mathsf{kg}) \leq \mathsf{t}_s[2] \cdot \mathsf{vol}_s(\mathsf{sid})\}$ ▷ t_s contains two values, one specifies the lower bound, and the other specifies the upper bound of the frequency range for the k-grams in the candidate set for a given s-term.

Refine the guesses using SIP

$\mathsf{change} \leftarrow \mathsf{True}$
while $\mathsf{change} = \mathsf{True}$ **do**
    $\mathsf{change} \leftarrow \mathsf{False}$
    for $(\mathsf{sid}_1, \mathsf{sid}_2, \Delta) \in \mathsf{SIP}$ **do**
        for $\mathsf{cand}_1 \in \mathsf{CandSet}_s(\mathsf{sid}_1)$ **do**
            $\mathsf{match} \leftarrow \mathsf{False}$
            for $\mathsf{cand}_2 \in \mathsf{CandSet}_s(\mathsf{sid}_2)$ **do**
                if $\Delta < 0 \wedge \mathsf{cand}_1[:\Delta] = \mathsf{cand}_2[-\Delta + 1:]$ **then**
                    $\mathsf{match} \leftarrow \mathsf{True}$
                if $\Delta > 0 \wedge \mathsf{cand}_1[\Delta + 1:] = \mathsf{cand}_2[:-\Delta]$ **then**
                    $\mathsf{match} \leftarrow \mathsf{True}$
            if $\mathsf{match} = \mathbf{True}$ **then**
                $\mathsf{CandSet}_s(\mathsf{sid}_1) \leftarrow \mathsf{CandSet}_s(\mathsf{sid}_1) \backslash \{\mathsf{cand}_1\}$
                $\mathsf{change} \leftarrow \mathsf{True}$

The procedure above is repeated with $(\mathsf{sid}_2, \mathsf{sid}_1, -\Delta)$. We omit it from the pseudocode for brevity.
return $\mathsf{CandSet}_s$

D.3 Attack Pseudocode For Our Attack Against the HLK Scheme

Algorithm 6 provides a high-level overview of our attack against the HLK scheme. The constituent components for generating candidate sets, trimming them, and then reconstructing the queries are detailed in Algorithms 7, 8 and 9, respectively.

E Derivation of the Likelihood Functions

E.1 Likelihood Function for the CS Scheme

In this section, We give a concrete derivation of the likelihood function used in our attack against the CS scheme. The likelihood function takes four inputs, namely:

- A guess $\mathsf{guess} : \mathbb{N} \to \Sigma$. The allocation $\mathsf{guess}(\mathsf{tk}) = a$ means that our guess for token

Algorithm 4 First round of simulated annealing

Parameters: Parameter for trimming the candidate sets for the x-terms t_x. The initial temperature T_{init}. The number of iterations of the first round of simulated annealing iter_1.

Input: Leakage representations $\mathsf{lk} = \{\mathsf{lk}_1, \ldots, \mathsf{lk}_n\}$, volume maps $(\mathsf{vol}_s, \mathsf{vol}_x)$, conditional intersection pattern SIP, candidate sets for the x-terms $\mathsf{CandSet}_s$, and auxiliary distributions $(\mathsf{Aux}_s, \mathsf{Aux}_x)$.

Output: Guesses for the s-terms $\mathsf{guess}_s : \mathbb{N} \to \Sigma^3$ that maps each s-term identifier to a k-gram (assuming $k = 3$). And Guesses for the x-terms $\mathsf{guess}_x : \mathbb{N} \to \Sigma^3$ that maps each x-term identifier to a k-gram.

Simulated_annealing1$(\mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), \mathsf{SIP}, \mathsf{CandSet}_s, (\mathsf{Aux}_s, \mathsf{Aux}_x))$

$T \leftarrow T_{init}$
$\mathsf{guess}_s \leftarrow$ **Initial_solution**$(\mathsf{CandSet}_s), \mathsf{guess}_x \leftarrow \{\}$
$L \leftarrow \{\}$
for $(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l))) \in \mathsf{lk}$ **do**
    $L(\mathsf{sid}) \leftarrow L(\mathsf{sid}) \cup \{(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l))\}$ ▷ Group the queries by their s-term support size.
for $\mathsf{iter} \in 1, \ldots, \mathsf{iter}_1$ **do**
    $D \leftarrow \{\}$ ▷ A map to keep track of the absolute distance between the x-terms and the s-terms that appears with the x-terms so far.
    $\mathsf{T} \leftarrow$ **Cooling**(T)
    for $\mathsf{sid} \in sorted(\{\mathsf{sid} \in L\}, key = \mathsf{vol}_s(\mathsf{sid}))$ **do** ▷ Sorting in decreasing order.
        $(\mathsf{guess}'_s, \mathsf{guess}'_x) \leftarrow$ **Neighbour1**$(L(\mathsf{sid}), \mathsf{CandSet}_s, \mathsf{vol}_x, \mathsf{Aux}_x, \mathsf{t}_x, D, (\mathsf{guess}_s, \mathsf{guess}_x))$
        $(\mathsf{guess}_s, \mathsf{guess}_x), D \leftarrow$ **Update_Assign**$((\mathsf{guess}_s, \mathsf{guess}_x), (\mathsf{guess}'_s, \mathsf{guess}'_x), \mathsf{T}, D, \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), (\mathsf{Aux}_s, \mathsf{Aux}_x))$
    return $(\mathsf{guess}_s, \mathsf{guess}_x)$

Initial_solution$(\mathsf{CandSet}_s)$

$\mathsf{guess}_s \leftarrow \{\}$
for $\mathsf{sid} \in \mathsf{CandSet}_s$ **do**
    $\mathsf{guess}_s(\mathsf{sid}) \xleftarrow{\$} \mathsf{CandSet}_s(\mathsf{sid})$
return guess_s

Neighbour1$(\{\mathsf{lk}_1, \ldots, \mathsf{lk}_m\}, \mathsf{CandSet}_s, \mathsf{vol}_x, \mathsf{Aux}_x, \mathsf{t}_x, D, (\mathsf{guess}_s, \mathsf{guess}_x)))$

$(\mathsf{guess}'_s, \mathsf{guess}'_x) \leftarrow (\mathsf{guess}_s, \mathsf{guess}_x)$
$(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l)) \leftarrow \mathsf{lk}_1$
$\mathsf{guess}'_s(\mathsf{sid}) \xleftarrow{\$} \mathsf{CandSet}_s(\mathsf{sid})$
for $j \in \{1, \ldots, m\}$ **do**
    $(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l)) \leftarrow \mathsf{lk}_j$
    for $k \leftarrow \{1, \ldots, j\}$ **do**
        if $\mathsf{xid}_k \notin D \ \vee D(\mathsf{xid}_k) > |\Delta_k|$ **then**
            $\mathsf{CandSet}_x \leftarrow \{\mathsf{kg} \mid \mathsf{t}_x[1] \cdot \mathsf{vol}_x(\mathsf{sid}, \Delta_k, \mathsf{xid}_k) \leq \mathsf{Aux}_x(\mathsf{guess}'_s(\mathsf{sid}), \Delta_k, \mathsf{kg}) \leq \mathsf{t}_x[2] \cdot \mathsf{vol}_x(\mathsf{sid}, \Delta_k, \mathsf{xid}_k)\}$
            $\mathsf{guess}'_x(\mathsf{xid}_k) \xleftarrow{\$} \mathsf{CandSet}_x$
return $\mathsf{guess}'_s, \mathsf{guess}'_x$

Update_Assign$((\mathsf{guess}_s, \mathsf{guess}_x), (\mathsf{guess}'_s, \mathsf{guess}'_x), \mathsf{T}, D, \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), (\mathsf{Aux}_s, \mathsf{Aux}_x))$

$\mathsf{sc} \leftarrow$ **Score**$((\mathsf{guess}_s, \mathsf{guess}_x), \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), (\mathsf{Aux}_s, \mathsf{Aux}_x))$
$\mathsf{sc'} \leftarrow$ **Score**$((\mathsf{guess}'_s, \mathsf{guess}'_x), \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), (\mathsf{Aux}_s, \mathsf{Aux}_x))$
if Accept_prob(sc, sc', T) $= 1$ **then**
    $(\mathsf{guess}_s, \mathsf{guess}_x) \leftarrow (\mathsf{guess}'_s, \mathsf{guess}'_x)$
    for $(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l))) \in L$ **do**
        for $k \in \{1, \ldots, l\}$ **do**
            $D(\mathsf{xid}_k) \leftarrow \min(D(\mathsf{xid}_k), |\Delta_k|)$
return $(\mathsf{guess}_s, \mathsf{guess}_x), D$

tk is letter a.

- Tokens $(\mathsf{tk}_{i,1}, \ldots, \mathsf{tk}_{i,m_i})_{i=1}^{l}$.
- A list of observed query response volumes $(\mathsf{vol}_i)_{i=1}^{l}$, indicating that the i-th query has returned vol_i matching indices.
- An auxiliary distribution $\mathsf{Aux} : \mathcal{P}(\Sigma) \to \mathbb{R}$. $\mathsf{Aux}(s) = r$ means that we model the distribution of the query response volume for string s as a Poisson distribution with rate r.

Algorithm 5 Second round of simulated annealing

Parameters: Parameter for trimming the candidate sets for the x-terms t_x. The initial temperature T_{init}. The number of iterations of the second round of simulated annealing iter_2.

Input: Guesses so far $(\mathsf{guess}_s, \mathsf{guess}_x)$, leakage representations $\mathsf{lk} = set\mathsf{lk}_1, \ldots, \mathsf{lk}_n$, volume maps $(\mathsf{vol}_s, \mathsf{vol}_x)$, conditional intersection pattern SIP, and auxiliary distributions $(\mathsf{Aux}_s, \mathsf{Aux}_x)$, s-term candidates $\mathsf{CandSet}_s$.

Output: Guesses for the s-terms $\mathsf{guess}_s : \mathbb{N} \to \Sigma^3$ that maps each s-term identifier to a k-gram (assuming $k = 3$). And Guesses for the x-terms $\mathsf{guess}_x : \mathbb{N} \to \Sigma^3$ that maps each x-term identifier to a k-gram.

Simulated_annealing2$((\mathsf{guess}_s, \mathsf{guess}_x), \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), \mathsf{SIP}, (\mathsf{Aux}_s, \mathsf{Aux}_x), \mathsf{CandSet}_s)$

$T \leftarrow T_{init}$
$L \leftarrow \{\}$
for $(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l))) \in \mathsf{lk}$ **do**
    $L(\mathsf{sid}) \leftarrow L(\mathsf{sid}) \cup \{(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l))\}$ ▷ Group the queries by their s-term support size.
for $\mathsf{iter} \in \{1, \ldots, \mathsf{iter}_2\}$ **do**
    $D \leftarrow \{\}$ ▷ A map to keep track of the absolute distance between the x-terms and the s-terms that appears with the x-terms so far.
    $\mathsf{T} \leftarrow$ **Cooling**(T)
    for $\mathsf{sid} \in sorted(\{\mathsf{sid} \in L\}, key = \mathsf{vol}_s(\mathsf{sid}))$ **do** ▷ Sorting in decreasing order.
        for $(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l))) \in L(\mathsf{sid})$ **do**
            $\mathsf{guess}'_s \leftarrow \mathsf{guess}_s$
            $\mathsf{guess}'_s(\mathsf{sid}) \xleftarrow{\$} \mathsf{CandSet}_s(\mathsf{sid})$
            $(\mathsf{guess}_s, \mathsf{guess}_x), D \leftarrow$ **Update_Assign**$((\mathsf{guess}_s, \mathsf{guess}_x), (\mathsf{guess}'_s, \mathsf{guess}_x), \mathsf{T}, D, \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), (\mathsf{Aux}_s, \mathsf{Aux}_x))$
            for $j \in \{1, \ldots, l\}$ **do**
                $\mathsf{guess}'_x \leftarrow$ **Neighbour_xterm**$(\mathsf{sid}, \Delta_j, \mathsf{xid}_j, \mathsf{vol}_x, \mathsf{Aux}_x, \mathsf{t}_x, D, (\mathsf{guess}_s, \mathsf{guess}_x))$
                $(\mathsf{guess}_s, \mathsf{guess}_x), D \leftarrow$ **Update_Assign**$((\mathsf{guess}_s, \mathsf{guess}_x), (\mathsf{guess}'_s, \mathsf{guess}_x), \mathsf{T}, D, \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x), (\mathsf{Aux}_s, \mathsf{Aux}_x))$
return $(\mathsf{guess}_s, \mathsf{guess}_x)$

Neighbour_xterm$(\mathsf{sid}, \Delta, \mathsf{xid}, \mathsf{vol}_x, \mathsf{Aux}_x, \mathsf{t}_x, D, (\mathsf{guess}_s, \mathsf{guess}_x)))$
$\mathsf{guess}'_x \leftarrow \mathsf{guess}_x$
if $\mathsf{xid} \notin D \ \vee D(\mathsf{xid}) > |\Delta|$ **then**
    $\mathsf{CandSet}_x \leftarrow \{\mathsf{kg}' \mid \mathsf{t}_x[1] \cdot \mathsf{vol}_x(\mathsf{sid}, \Delta_k, \mathsf{xid}_k) \leq \mathsf{Aux}_x(\mathsf{kg}, \Delta_k, \mathsf{kg}') \leq \mathsf{t}_x[2] \cdot \mathsf{vol}_x(\mathsf{sid}, \Delta_k, \mathsf{xid}_k)\}$
    $\mathsf{guess}'_x(\mathsf{xid}) \xleftarrow{\$} \mathsf{CandSet}_x$
return guess'_x

We begin by writing down the probability of observing the query response volumes given the guess. The tokens and the auxiliary distribution will show up as constants in the expression.

$$\mathbf{Pr}[(\mathsf{vol}_i)_{i=1}^{l} \mid \mathsf{guess}] = \prod_{i=1}^{l} \mathbf{Pr}[\mathbf{Pois}(\mathsf{Aux}((\mathsf{guess}(\mathsf{tk}_{i,1}), \ldots, \mathsf{guess}(\mathsf{tk}_{i,m_i})))) = \mathsf{vol}_i]$$

In the expression, $s = (\mathsf{guess}(\mathsf{tk}_{i,1}), \ldots, \mathsf{guess}(\mathsf{tk}_{i,m_i}))$ is the guess we have for the i-th query (as a string). $\mathsf{Aux}(s)$ gives the rate of the string in the auxiliary information and

Algorithm 6 Attack against the HLK scheme

Parameters: Parameters for trimming the candidate sets δ and σ, the number of iterations $\mathsf{iter}_{\mathsf{max}}$

Input: Leakage from the queries $\mathsf{lk} = (\mathsf{lk}_1, \ldots, \mathsf{lk}_n)$ (the queries are ordered by increasing number of tokens), auxiliary information $\mathsf{rank}, \mathsf{vol}, \mathsf{Aux}_3$, and $\mathsf{Aux}_{\leq 3}$.

Output: Guesses of the plaintext queries guess.

Attack$(\mathsf{lk}, \mathsf{rank}, \mathsf{vol}, \mathsf{Aux}_3, \mathsf{Aux}_{\leq 3})$

$\mathsf{CandSet} \leftarrow \emptyset$
for $\mathsf{iter} \in \{1, \ldots, \mathsf{iter}_{\mathsf{max}}\}$ **do**
  $\mathsf{CandSet} \leftarrow$ **Gen_Candidates**$(\mathsf{CandSet}, \mathsf{lk}, \mathsf{rank}, \mathsf{vol}, \delta, \sigma)$
  $\mathsf{CandSet}' \leftarrow \emptyset$
  while $\mathsf{CandSet}' \neq \mathsf{CandSet}$ **do**
    $\mathsf{CandSet} \leftarrow \mathsf{CandSet}'$
    $\mathsf{CandSet}' \leftarrow$ **Trim_Candidates**$(\mathsf{CandSet}, \mathsf{lk}, \mathsf{Aux}_3, \mathsf{Aux}_{\leq 3})$
$\mathsf{guess} \leftarrow$ **Reconstruct_Queries**$(\mathsf{CandSet}, \mathsf{lk})$
return guess

Algorithm 7 Generate the candidates

Gen_Candidates$(\mathsf{CandSet}, \mathsf{lk}, \mathsf{rank}, \mathsf{vol}, \delta, \sigma)$

$\mathsf{incst} \leftarrow \emptyset$
for $\mathsf{kid} \in \mathsf{CandSet}.keys()$ **do**
  if $|\mathsf{CandSet}(\mathsf{kid})[2]| = 1$ **then**
    $\mathsf{incst}(\mathsf{kid}) \leftarrow 0$
for $\mathsf{kid}_1 \in \mathsf{CandSet}.keys()$ **do**
  for $\mathsf{kid}_2 \in \mathsf{CandSet}.keys()$ **do**
    if $|\mathsf{CandSet}(\mathsf{kid}_1)[2]| = 1 \wedge |\mathsf{CandSet}(\mathsf{kid}_2)[2]| = 1 \wedge \mathsf{CandSet}(\mathsf{kid}_1)[1] < \mathsf{CandSet}(\mathsf{kid}_2)[1] \wedge \mathsf{CandSet}(\mathsf{kid}_1)[2][1] > \mathsf{CandSet}(\mathsf{kid}_2)[2][1]$ **then**
      $\mathsf{incst}(\mathsf{kid}_1) \leftarrow \mathsf{incst}(\mathsf{kid}_1) + 1, \mathsf{incst}(\mathsf{kid}_2) \leftarrow \mathsf{incst}(\mathsf{kid}_2) + 1$
    if $|\mathsf{CandSet}(\mathsf{kid}_1)[2]| = 1 \wedge |\mathsf{CandSet}(\mathsf{kid}_2)[2]| = 1 \wedge \mathsf{CandSet}(\mathsf{kid}_1)[1] > \mathsf{CandSet}(\mathsf{kid}_2)[1] \wedge \mathsf{CandSet}(\mathsf{kid}_1)[2][1] < \mathsf{CandSet}(\mathsf{kid}_2)[2][1]$ **then**
      $\mathsf{incst}(\mathsf{kid}_1) \leftarrow \mathsf{incst}(\mathsf{kid}_1) + 1, \mathsf{incst}(\mathsf{kid}_2) \leftarrow \mathsf{incst}(\mathsf{kid}_2) + 1$
$\mathsf{incst\_kids} \leftarrow sorted(\{\mathsf{kid} \in \mathsf{incst}\}, key = \mathsf{incst}(\mathsf{kid}))$ ▷ Sorting in decreasing order
$\mathsf{CandSet}' \leftarrow \emptyset$
for $\mathsf{lk}_i \in \mathsf{lk}$ **do**
  $\{(\mathsf{kid}_1, \mathsf{idx}_{1,1}, \mathsf{idx}_{1,2}), \ldots, (\mathsf{kid}_l, \mathsf{idx}_{l,1}, \mathsf{idx}_{l,2})\} \leftarrow \mathsf{lk}_i$
  for $j \in \{1, \ldots, l\}$ **do**
    if $\mathsf{kid}_j \in \mathsf{incst\_kids}[\lfloor 0.1 \cdot |\mathsf{incst\_kids}| \rfloor :]$ **then**
      $\mathsf{CandSet}'(\mathsf{kid}_j) \leftarrow \mathsf{CandSet}(\mathsf{kid}_j)$
    else
      $\mathsf{sd} \leftarrow \sqrt{(\mathsf{idx}_{j,2} - \mathsf{idx}_{j,1} + 1) \cdot (N - (\mathsf{idx}_{j,2} - \mathsf{idx}_{j,1} + 1))/N}$ ▷ N is the number of k-grams in all documents
      $\mathsf{CandSet}'(\mathsf{kid}_j) \leftarrow \{\mathsf{kg} \mid \mathsf{idx}_{j,1} - \delta \cdot N \leq \mathsf{rank}(\mathsf{kg}) \leq \mathsf{idx}_{j,1} + \delta \cdot N, (\mathsf{idx}_{j,2} - \mathsf{idx}_{j,1} + 1) - \sigma \cdot \mathsf{sd} \leq \mathsf{vol}(\mathsf{kg}) \leq (\mathsf{idx}_{j,2} - \mathsf{idx}_{j,1} + 1) + \sigma \cdot \mathsf{sd}\}$
return $\mathsf{CandSet}'$

$\mathbf{Pr}[\mathbf{Pois}(\mathsf{Aux}(s)) = \mathsf{vol}_i]$ computes the probability of q_i having query response volume vol_i given the guess. Finally, the product in the end is due to our assumption on the independence of the distributions of the individual query response volumes.

Algorithm 8 Trim the candidates

Trim_Candidates($\mathsf{CandSet}, \mathsf{lk}, \mathsf{Aux}_3, \mathsf{Aux}_{\leq 3}$)

$\mathsf{CandSet}' \leftarrow \mathsf{CandSet}$
for $\mathsf{lk}_i \in \mathsf{lk}$ **do**
  $\{(\mathsf{kid}_1, \mathsf{idx}_{1,1}, \mathsf{idx}_{1,2}), \ldots, (\mathsf{kid}_l, \mathsf{idx}_{l,1}, \mathsf{idx}_{l,2})\} \leftarrow \mathsf{lk}_i$
  $q \leftarrow \emptyset$
  for $(j_1, \ldots, j_l) \in Perm(l)$ **do** ▷ $Perm(l)$ produces all permutations of $\{1, \ldots, l\}$
    $q_1 \leftarrow \{((\mathsf{CandSet}(\mathsf{kid}_{j_1})), j_1)\}$
    for $k \in \{2, \ldots, l-1\}$ **do**
      $q_k \leftarrow \{((\mathsf{kg}_1, \ldots, \mathsf{kg}_k), (j_1, \ldots, j_k)) \mid ((\mathsf{kg}_1, \ldots, \mathsf{kg}_{k-1}), (j_1, \ldots, j_{k-1})) \in q_{k-1}, (\mathsf{kg}_{k-1}, \mathsf{kg}_k) \in \mathsf{Aux}_3\}$
    $q_l \leftarrow \{((\mathsf{kg}_1, \ldots, \mathsf{kg}_l), (j_1, \ldots, j_l)) \mid ((\mathsf{kg}_1, \ldots, \mathsf{kg}_{l-1}), (j_1, \ldots, j_{l-1})) \in q_{l-1}, (\mathsf{kg}_{l-1}, \mathsf{kg}_l) \in \mathsf{Aux}_{\leq 3}\}$
    $q \leftarrow q \cup q_l$
  if $|q| = 1$ **then**
    $((\mathsf{kg}_1, \ldots, \mathsf{kg}_l), (j_1, \ldots, j_l)) \leftarrow q[1]$
    for $i \in \{1, \ldots, l\}$ **do**
      $\mathsf{CandSet}'(\mathsf{kg}_{j_i}) \leftarrow \{\mathsf{kg}_i\}$
return $\mathsf{CandSet}'$

Algorithm 9 Reconstruct the queries

Reconstruct_Queries($\mathsf{CandSet}, \mathsf{lk}$)

$\mathsf{guess} \leftarrow \emptyset$
for $\mathsf{lk}_i \in \mathsf{lk}$ **do**
  $\{(\mathsf{kid}_1, \mathsf{idx}_{1,1}, \mathsf{idx}_{1,2}), \ldots, (\mathsf{kid}_l, \mathsf{idx}_{l,1}, \mathsf{idx}_{l,2})\} \leftarrow \mathsf{lk}_i$
  $q \leftarrow \emptyset$
  for $(j_1, \ldots, j_l) \in Perm(l)$ **do** ▷ $Perm(l)$ produces all permutations of $\{1, \ldots, l\}$
    $q_1 \leftarrow \{(\mathsf{CandSet}(\mathsf{kid}_{j_1}))\}$
    for $k \in \{2, \ldots, l-1\}$ **do**
      $q_k \leftarrow \{(\mathsf{kg}_1, \ldots, \mathsf{kg}_k) \mid (\mathsf{kg}_1, \ldots, \mathsf{kg}_{k-1}) \in q_{k-1}, (\mathsf{kg}_{k-1}, \mathsf{kg}_k) \in \mathsf{Aux}_3\}$
    $q_l \leftarrow \{(\mathsf{kg}_1, \ldots, \mathsf{kg}_l) \mid (\mathsf{kg}_1, \ldots, \mathsf{kg}_{l-1}) \in q_{l-1}, (\mathsf{kg}_{l-1}, \mathsf{kg}_l) \in \mathsf{Aux}_{\leq 3}\}$
    $q \leftarrow q \cup q_l$
  if $|q| = 1$ **then**
    $\mathsf{guess}[i] \leftarrow q[1]$
return *guess*

Given the probability expression, we can use Bayes' theorem to turn the probability into a likelihood.

$$\begin{aligned}\mathbf{L}[\mathsf{guess} \mid (\mathsf{vol}_i)_{i=1}^l] =& \frac{\mathbf{Pr}[\mathsf{guess}] \cdot \mathbf{Pr}[(\mathsf{vol}_i)_{i=1}^l \mid \mathsf{guess}]}{\mathbf{Pr}[(\mathsf{vol}_i)_{i=1}^l]} \\ \propto& \mathbf{Pr}[\mathsf{guess}] \cdot \mathbf{Pr}[(\mathsf{vol}_i)_{i=1}^l \mid \mathsf{guess}]\end{aligned}$$

We assume that all guesses are equally likely in our attack, so the likelihood function is proportional to $\mathbf{Pr}[(\mathsf{vol}_i)_{i=1}^l \mid \mathsf{guess}]$.

One caveat is that if $s = (\mathsf{guess}(\mathsf{tk}_{i,1}), \ldots, \mathsf{guess}(\mathsf{tk}_{i,m_i}))$ is not in Aux, then $\mathbf{Pr}[\mathbf{Pois}(\mathsf{Aux}(s)) = \mathsf{vol}_i]$ will be zero. This will make the whole likelihood function zero as well, due to its multi-

plicative nature. In practice, as the search space is sparse (meaning that only a small number of guesses yield non-zero likelihood), having zero in some of the multiplicative terms is unavoidable. On the other hand, we want to avoid these cases since the search algorithm we use depends on comparing the likelihoods of different guesses (zero is not helpful for this). For this reason, we use Laplace smoothing (with $\alpha = 1$) whenever $\mathsf{Aux}(s) = 0$. In addition, we use the log of the likelihood in our implementation to maintain accuracy in the otherwise very small numbers.

E.2 Likelihood Function for the FJK+ Scheme

In this section, We give a concrete derivation of the likelihood function used in our attack against the FJK+ scheme. The likelihood function takes four inputs, namely:

- The guess for the s-terms and x-terms $(\mathsf{guess}_s, \mathsf{guess}_x)$. $\mathsf{guess}_s : \mathbb{N} \to \Sigma^3$ maps each s-term identifier to a k-gram (assuming $k = 3$). $\mathsf{guess}_x : \mathbb{N} \to \Sigma^3$ maps each x-term identifier to a k-gram.
- The leakage representation $\mathsf{lk} = \{\mathsf{lk}_1, \ldots, \mathsf{lk}_n\}$. Each entry of lk has the shape
$$(\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l))).$$
- The volume maps (as a part of the leakage) $(\mathsf{vol}_s, \mathsf{vol}_x)$. $\mathsf{vol}_s : \mathbb{N} \to \mathbb{N}$ maps each s-term identifier to their s-term support size. $\mathsf{vol}_x : \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ maps an s-term identifier sid, an offset Δ, and an x-term identifier xid to the results pattern size of the query corresponding to $(\mathsf{sid}, (\Delta, \mathsf{xid}))$.
- The auxiliary distributions $(\mathsf{Aux}_s, \mathsf{Aux}_x)$. Aux_s maps n-grams to their expected count in the database (after normalisation). Aux_x maps an s-term, an offset, and an x-term to its expected count in the database (after normalisation).

The likelihood function also has access to N, the total number of k-grams in all documents as a constant.

Similar to the likelihood function for our attack against the CS scheme, we can write the likelihood function as:

$$\begin{aligned}&\mathbf{L}[(\mathsf{guess}_s, \mathsf{guess}_x) \mid \mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x)] \\ \propto &\mathbf{Pr}[(\mathsf{guess}_s, \mathsf{guess}_x)] \cdot \mathbf{Pr}[\mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x) \mid (\mathsf{guess}_s, \mathsf{guess}_x)].\end{aligned}$$

In particular, the probability $\mathbf{Pr}[\mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x) \mid \mathsf{guess}]$ can be further decomposed into:

$$\begin{aligned}&\mathbf{Pr}[\mathsf{lk}, (\mathsf{vol}_s, \mathsf{vol}_x) \mid (\mathsf{guess}_s, \mathsf{guess}_x)] \\ = &\prod_{i=1}^{n} \mathbf{Pr}[\mathsf{lk}_i, (\mathsf{vol}_s, \mathsf{vol}_x) \mid (\mathsf{guess}_s, \mathsf{guess}_x)].\end{aligned}$$

Suppose $\mathsf{lk}_i = (\mathsf{sid}, (\Delta_1, \mathsf{xid}_1), \ldots, (\Delta_l, \mathsf{xid}_l)))$. Then the terms in the product can be written as:

$$\begin{aligned}
&\mathbf{Pr}[\mathsf{lk}_i, (\mathsf{vol}_s, \mathsf{vol}_x) \mid (\mathsf{guess}_s, \mathsf{guess}_x)] \\
=&\mathbf{Pr}[\mathbf{Pois}(\mathsf{Aux}_s(\mathsf{guess}_s(\mathsf{sid}))) = \mathsf{vol}_s(\mathsf{sid})] \cdot \\
&\prod_{j=1}^{l} \mathbf{Pr}[\mathbf{Pois}(\mathsf{Aux}_x(\mathsf{guess}_s(\mathsf{sid}), \Delta_j, \mathsf{guess}_x(\mathsf{xid}_j))) = \mathsf{vol}_x(\mathsf{xid}_j)].
\end{aligned}$$

Similar to our attack against the CS scheme, we apply Laplace smoothing whenever the input to Aux_s or Aux_x is not found. In addition, we use the log of the likelihood in our implementation to maintain accuracy in the otherwise very small numbers.

F Description of the Datasets

Simple English Wikipedia. Simple English Wikipedia is a collection of Wikipedia pages in Simple English words and grammar. There are over 220K Simple English Wikipedia pages. We used the dump of the Simple English Wikipedia from November 2025 in our experiments. We treat the text (after converting them to the lower case) in each Wikipedia page as a string.

Enron Email Corpus. The Enron Email Corpus is a collection of emails from 158 employees of the Enron Corporation in the years leading up to the company's collapse in December 2001. The corpus was generated from Enron email servers by the Federal Energy Regulatory Commission (FERC) during its subsequent investigation. Since these emails come from real people, we treat them with due care. See the section on Ethical Considerations for further discussion.